

LLM-based Agents and Prompt Optimization

Instructor: Hyunwoo J. Kim

News from Curriculum Committee

- The Course ID (CS407000) will change to CS3XXXXX from the next semester.
- The content/materials will change a little bit to make it as a 3XX-level course.
- Unfortunately, it would be administratively difficult to retake this course.
- Also, a graduate student cannot take this course.

Today's agenda

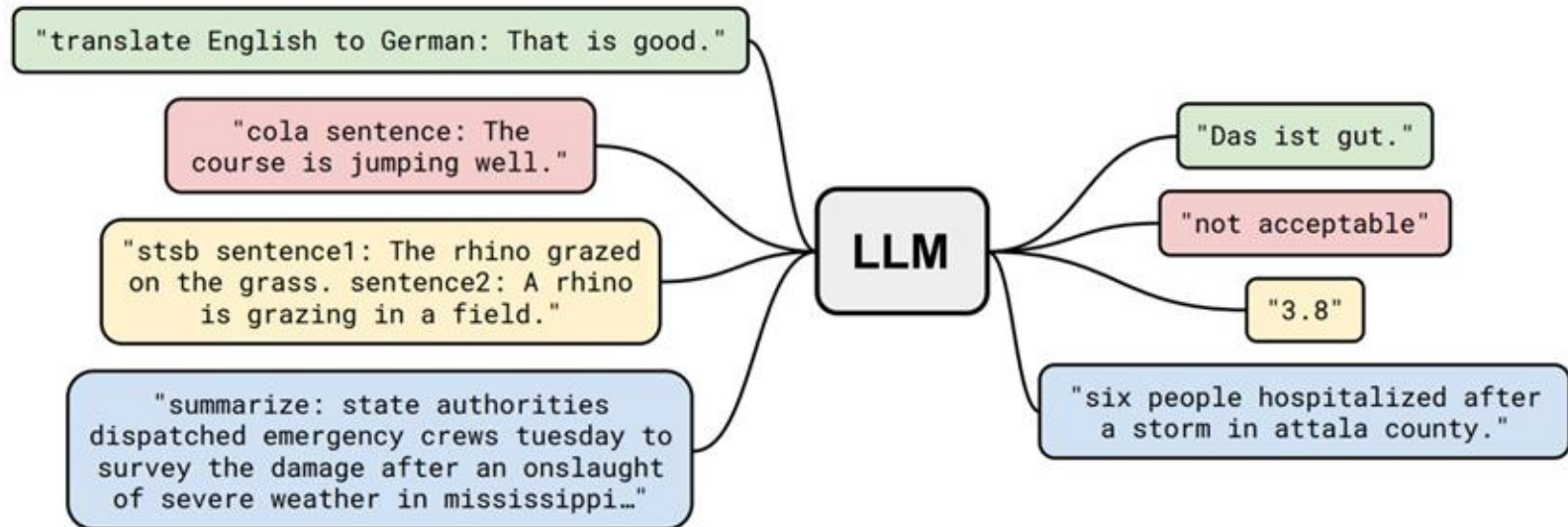
Attendance Code:

1. Prompt Engineering History
2. Soft Prompt Learning
3. LLM-based Hard Prompt Optimization
4. System Prompts for LLM Agents System
5. Prompts for Multi Agent System (MAS)

1. Prompt Engineering History

What is Prompt Engineering?

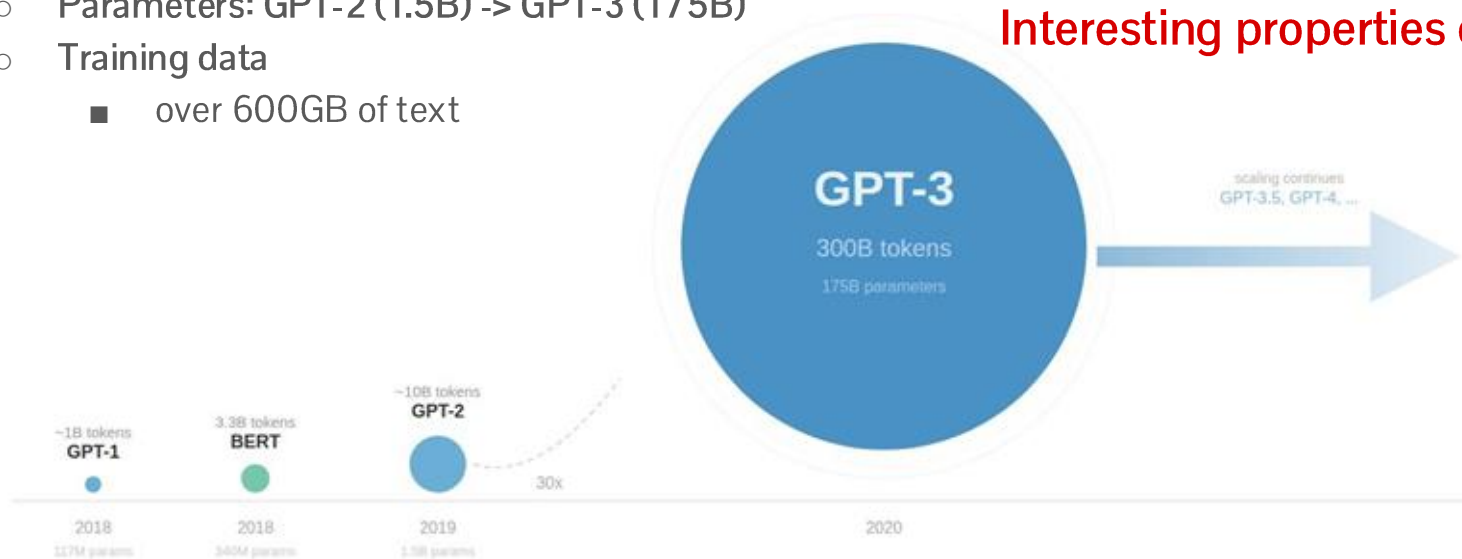
Prompt Engineering: The practice of systematically improving prompts for LLM applications-through testing, evaluation, analysis and optimization of prompts



Emergent abilities of large language models: GPT-3

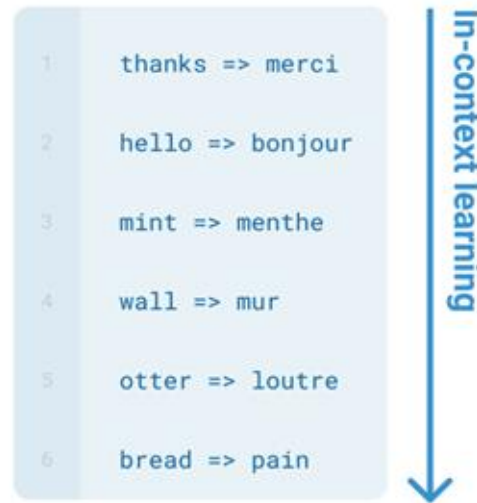
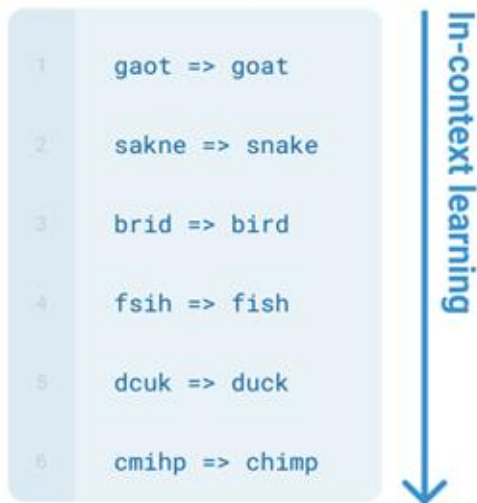
GPT-3

- Huge increase in size and training data
 - Parameters: GPT-2 (1.5B) -> GPT-3 (175B)
 - Training data
 - over 600GB of text



Emergent few shot learning

- Specify a task by simply prepending examples of the task before your example
- Also called **in-context learning**, to stress that no gradient updates are performed when learning a new task (there is a separate literature on few-shot learning with gradient updates)



Emergent few shot learning

Zero-shot

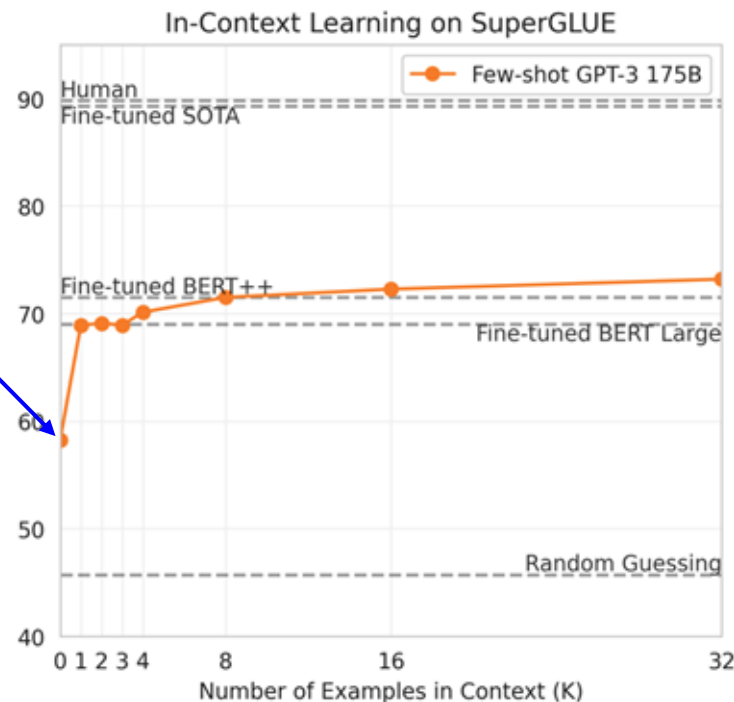
The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

1 Translate English to French:

← task description

2 cheese =>

← prompt

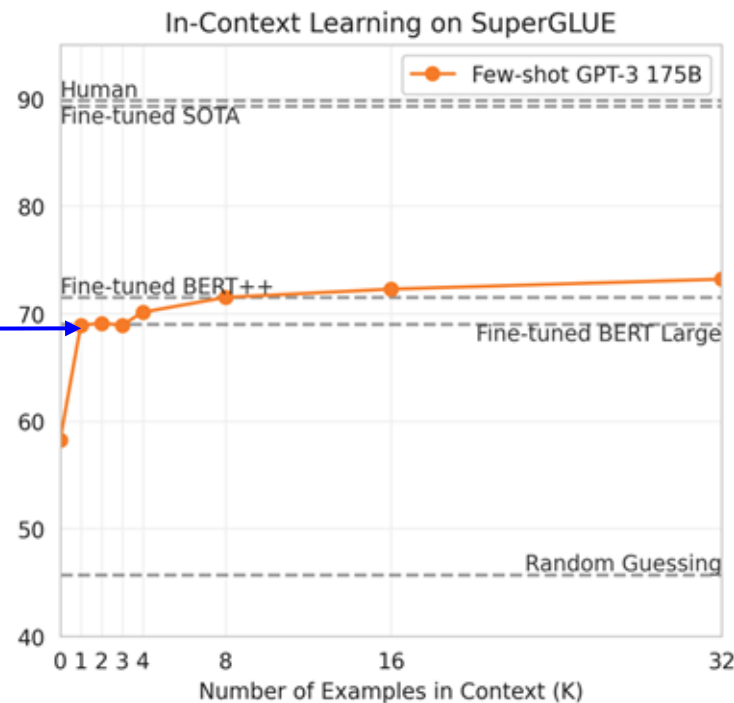


Emergent few shot learning

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← example
3	cheese =>	← prompt

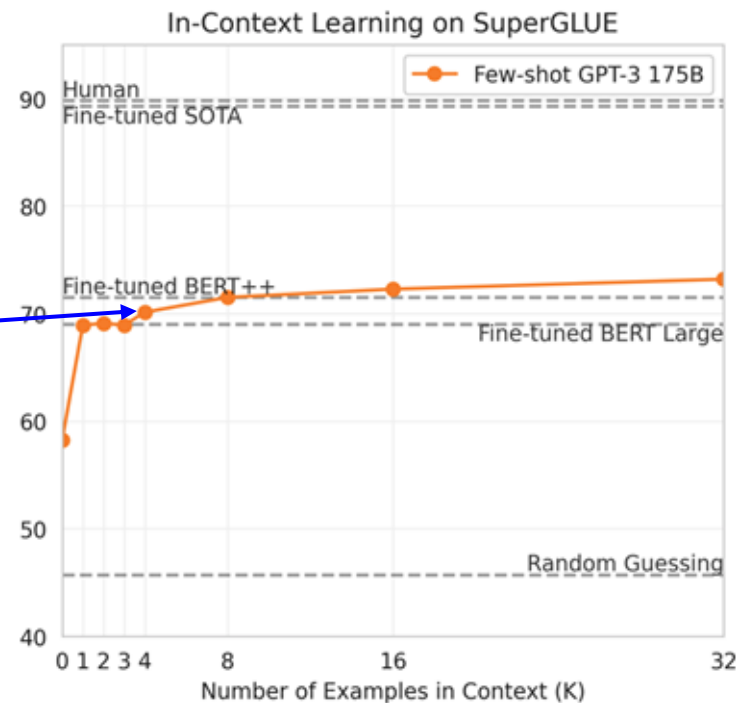


Emergent few shot learning

Few-shot

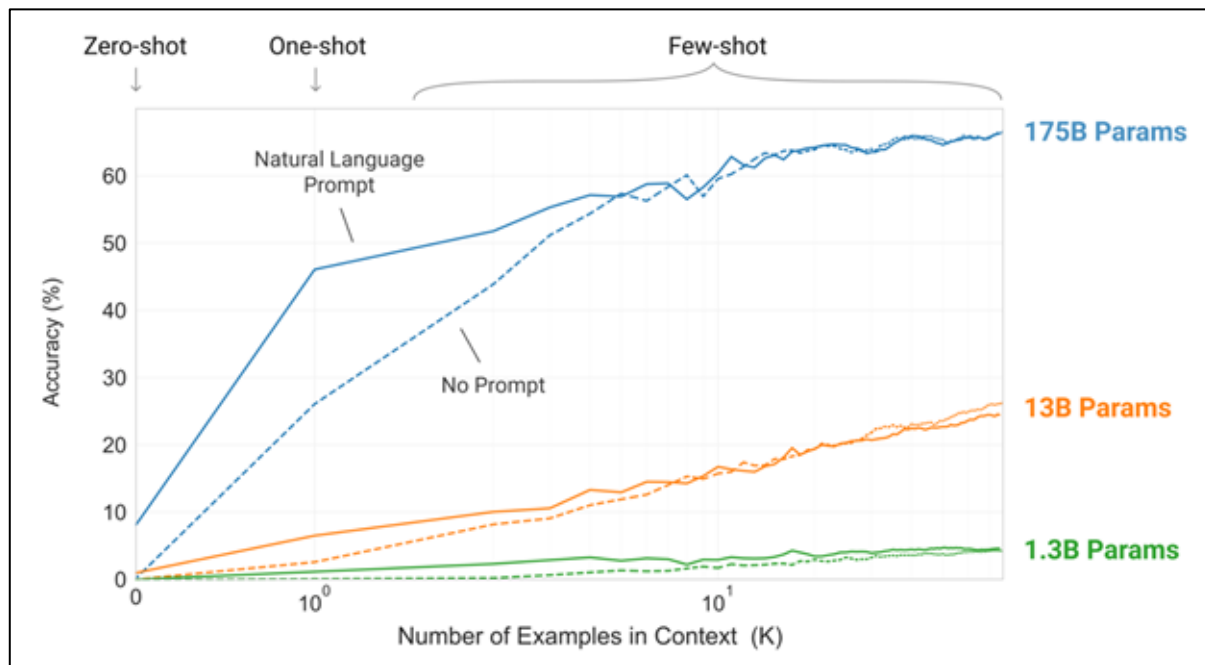
In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← examples
3	peppermint => menthe poivrée	←
4	plush girafe => girafe peluche	←
5	cheese =>	← prompt



Few-shot learning - Emergent property of model scale

Attendance Code:



Larger language models can learn tasks directly from context/examples without gradient updates.

Limitations of Few-shot learning?

- Some tasks seem too hard for even large LMs to learn through few-shot prompting alone
- Especially, tasks involving richer, multi-step reasoning

19583 + 29534 = 49117
98394 + 49384 = 147778
29382 + 12347 = 41729
93847 + 39299 = ?

Solution: Change the prompt!

Chain-of-thought (CoT) prompting

Chain-of-thought: a series of intermediate **reasoning** steps

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

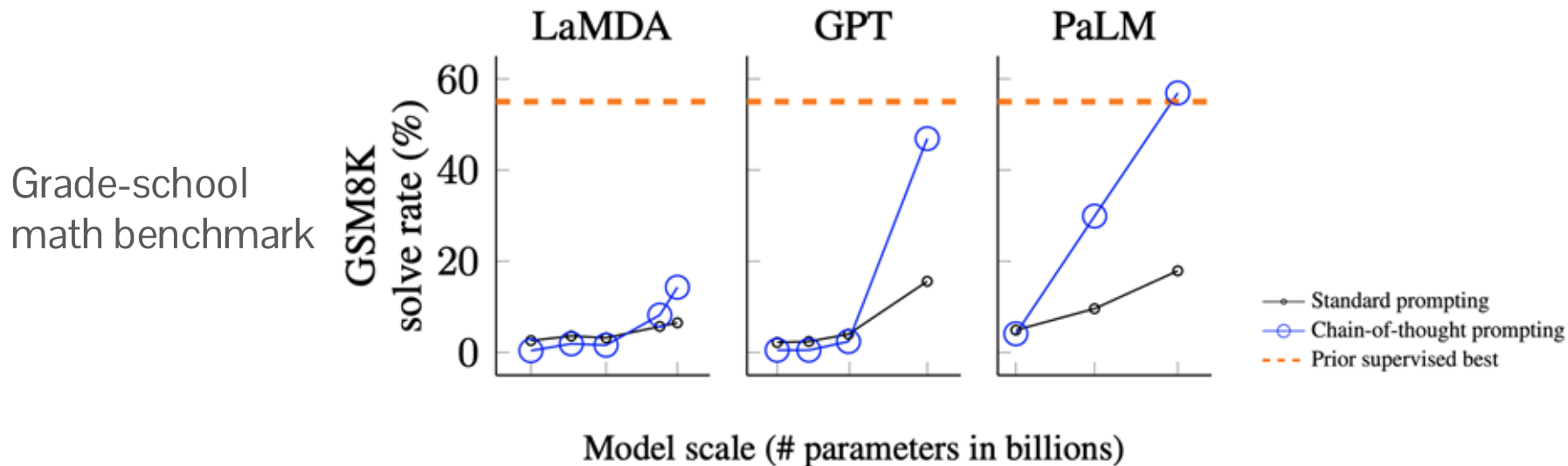
Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain-of-thought prompting

- Chain-of-thought prompting improves LLMs ability to solve math problems
- Emergent ability of increasing model scale.



Chain-of-thought prompting

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

Do we even need examples of reasoning?
Can we just ask the model to reason through things?

Zero-shot Chain-of-thought prompting

Simply add “**Let’s think step by step**” before each answer!

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓

Few-shot CoT

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let’s think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Zero-shot-CoT

Zero-shot Chain-of-thought prompting

	MultiArith	GSM8K
Zero-Shot	17.7	10.4
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	78.7	40.7
Few-Shot-CoT (2 samples)	84.8	41.3
Few-Shot-CoT (4 samples : First) (*1)	89.2	-
Few-Shot-CoT (4 samples : Second) (*1)	90.5	-
Few-Shot-CoT (8 samples)	93.0	48.7

Greatly outperforms zero-shot

Manual CoT is still better

Zero-shot Chain-of-thought prompting

- Performance is highly sensitive to prompts!
- Need manual effort to search for good hand-crafted prompts

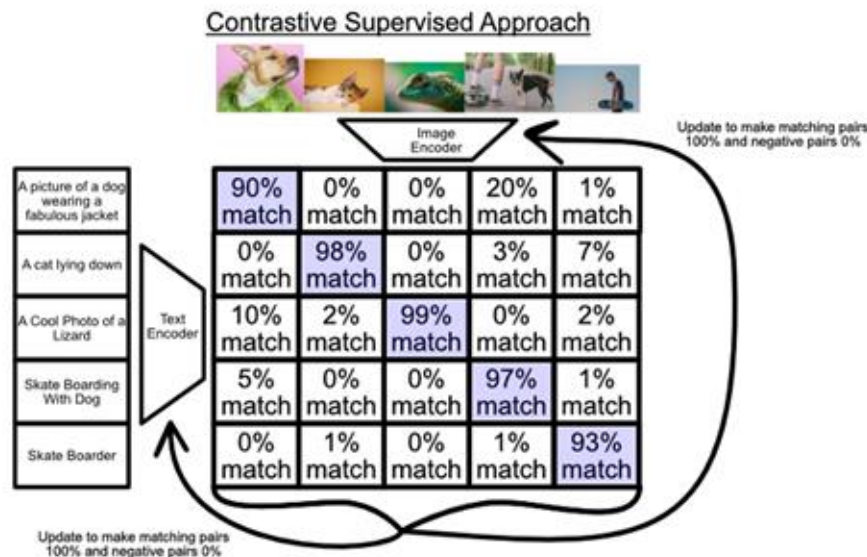
No.	Category	Template	Accuracy
1	instructive	Let's think step by step.	78.7
2		First, (*1)	77.3
3		Let's think about this logically.	74.5
4		Let's solve this problem by splitting it into steps. (*2)	72.2
5		Let's be realistic and think step by step.	70.8
6		Let's think like a detective step by step.	70.3
7		Let's think	57.5
8		Before we dive into the answer,	55.7
9		The answer is after the proof.	45.7

2. Soft Prompt Learning

CLIP (Contrastive Language-Image Pre-training)

- Vision-Language Model (VLM)
- Learning a joint embedding space for two modalities
- Contrastive loss

$$p(y|\mathbf{x}) = \frac{\exp(\text{sim}(\mathbf{z}, \mathbf{w}_y) / \tau)}{\sum_{j=1}^{N_c} \exp(\text{sim}(\mathbf{z}, \mathbf{w}_j) / \tau)}$$



Learning to Prompt

- Identifying the right prompt is a non-trivial task
 - A slight change in wording could make a huge difference in performance.

CLIP Image
classification on
Caltech101

Caltech101	Prompt	Accuracy
	a [CLASS].	82.68
	a photo of [CLASS].	80.81
	a photo of a [CLASS].	86.29

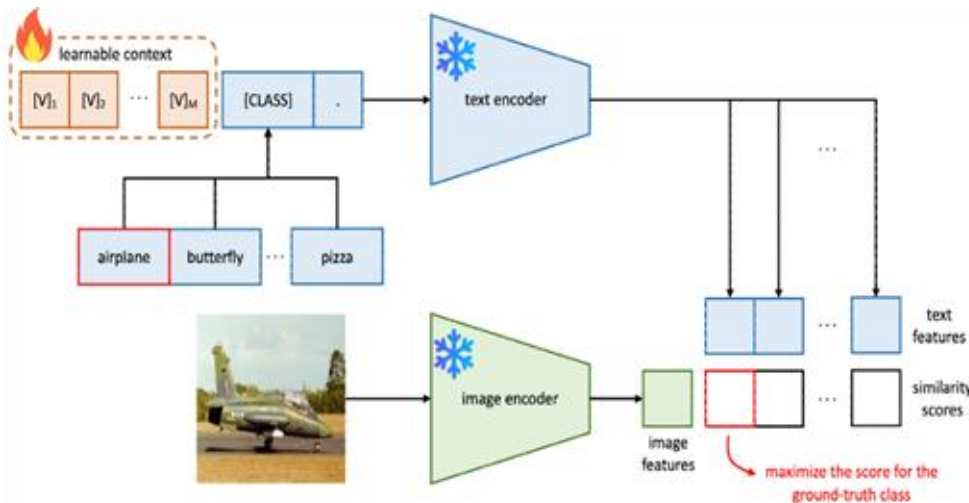
adding “a” brings more than 5% increase in performance

- Prompt engineering also requires prior knowledge about the task

Can we automate prompt engineering?

CoOp (Context Optimization)

Model a prompt's context by learning soft prompts (*a set of learnable vectors*)



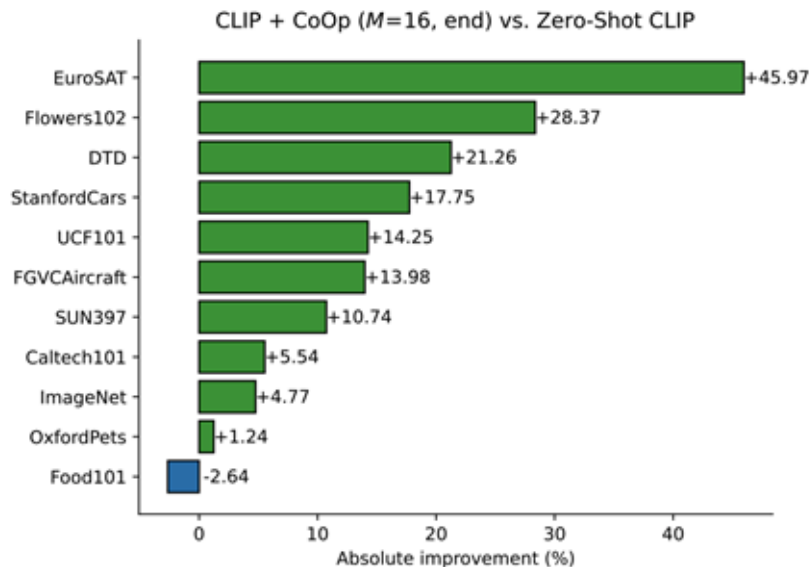
[Training Objectives]

- Minimizing the classification loss based on cross-entropy

$$p(y = i|x) = \frac{\exp(\cos(g(t_i), f)/\tau)}{\sum_{j=1}^K \exp(\cos(g(t_j), f)/\tau)}$$

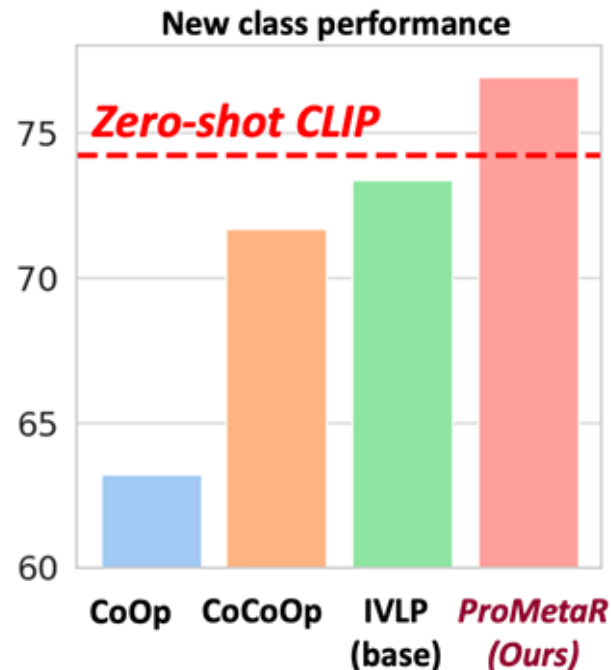
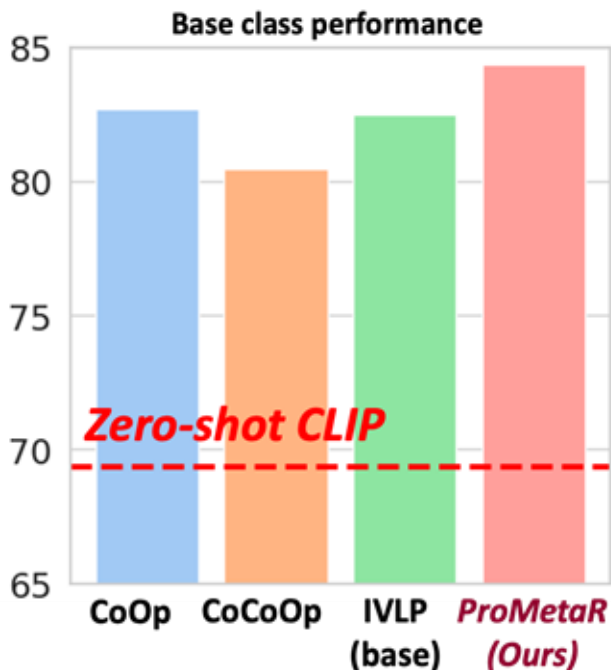
CoOp (Context Optimization)

In few shot setting(training on few data), CoOp outperforms handcrafted prompts on many image classification tasks.



Related works from **MLV** - ProMetaR

Most existing prompt learning methods suffer from task overfitting, limiting generalization



ProMetaR

ProMetaR solves it by bi-level optimization

- **Outer-loop:** Learn how much to regularize the prompt so that it generalizes well to validation data
- **Inner-loop:** Learn a task-adapted prompt from the training data

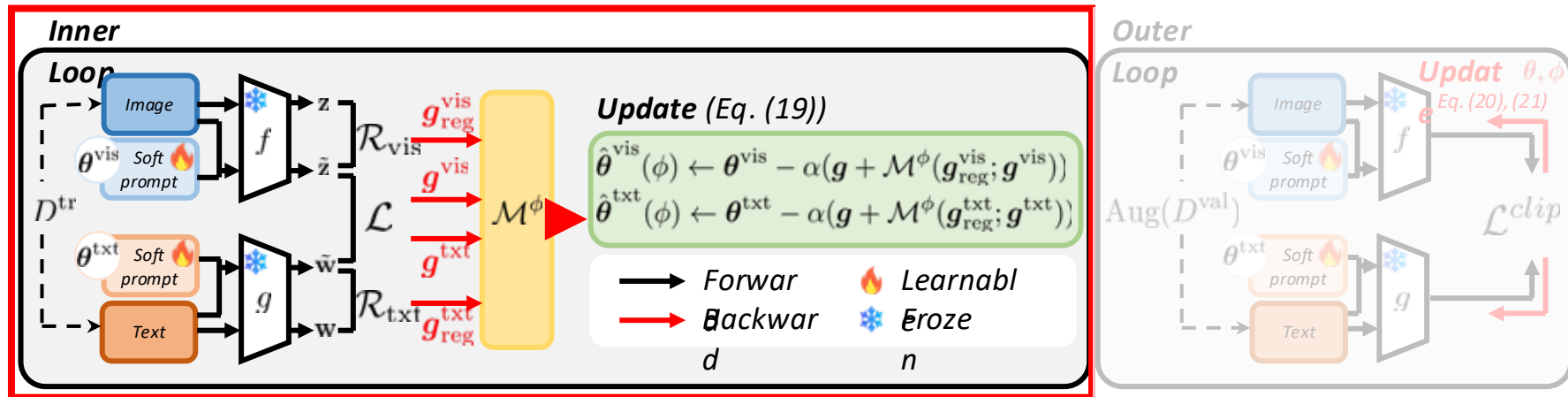
$$\min_{\Theta, \phi} \mathcal{L} \left(\Theta^* (\phi); D^{\text{val}} \right)$$

$$\Theta^* (\phi) = \arg \min_{\Theta} \mathcal{L} \left(\Theta; D^{\text{tr}} \right) + \mathcal{R}^{\phi} \left(\Theta; D^{\text{tr}} \right)$$

$\mathcal{L} \left(\Theta; D^{\text{tr}} \right)$: downstream loss (contrastive loss in CLIP) that focuses on ***task-specific knowledge***

$\mathcal{R}^{\phi} \left(\Theta; D^{\text{tr}} \right)$: regularizer loss that captures **task-agnostic general knowledge**.

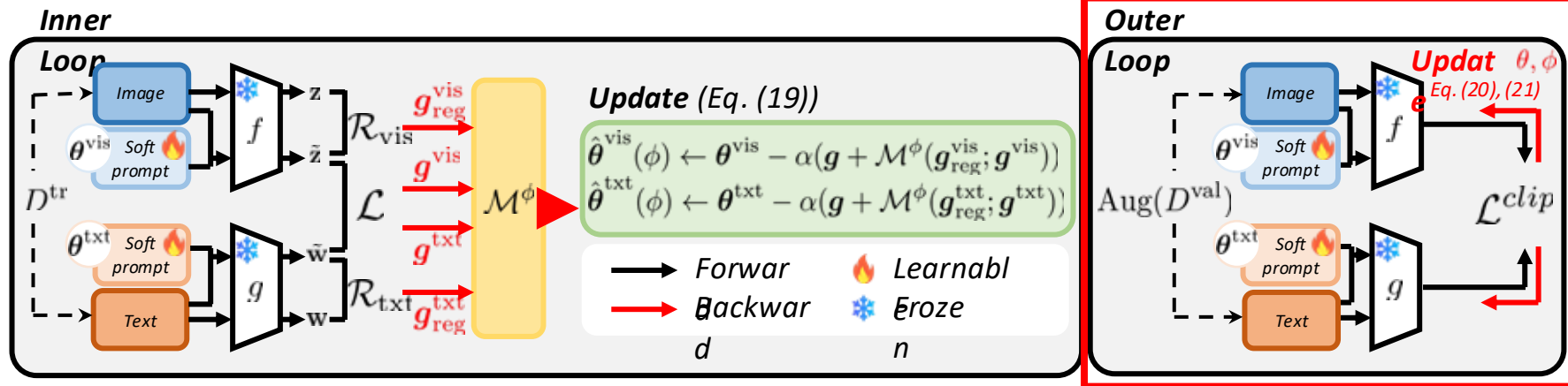
ProMetaR



Inner-loop: Update soft prompts with gradients induced by the contrastive loss and modulated gradients from the regularizer.

$$\hat{\Theta}(\phi) = \Theta - \alpha \left(g + \mathcal{M}^\phi(g_{\text{reg}}; g) \right) \quad g = \nabla_{\Theta} \mathcal{L}(\Theta; D^{\text{tr}}) \quad g_{\text{reg}} = \nabla_{\Theta} \mathcal{R}(\Theta; D^{\text{tr}})$$

ProMetaR

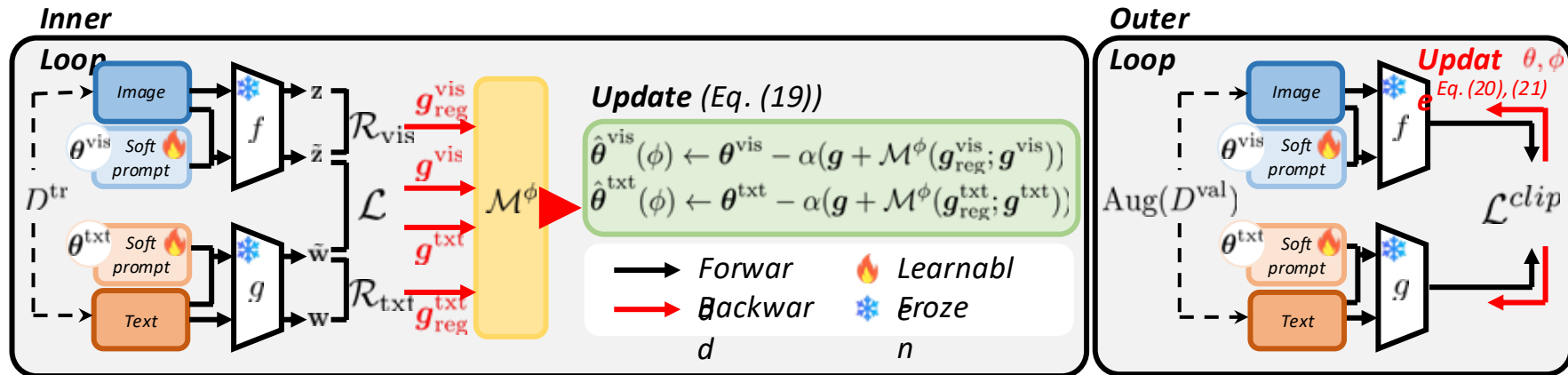


Outer-loop: After the update, the soft prompts and gradient modulation function are optimized for the validation set.

$$\Theta \leftarrow \Theta - \beta \nabla_{\Theta} \mathcal{L}(\hat{\Theta}(\phi); Aug(D^{val}))$$

$$\phi \leftarrow \phi - \beta \nabla_{\phi} \mathcal{L}(\hat{\Theta}(\phi); Aug(D^{val}))$$

ProMetaR



ProMetaR generates new labels to augment many tasks (data), encouraging the parameters to be optimized for diverse tasks (data):

$$\hat{h}_{val}^{(i)} = \rho h_{val}^{(i)} + (1 - \rho) h_{tr}^{(j)}, \quad \hat{y}_{val}^{(i)} = \rho y_{val}^{(i)} + (1 - \rho) y_{tr}^{(j)},$$

Related works from MLV - RPO

RPO (Read-only Prompt Optimization)

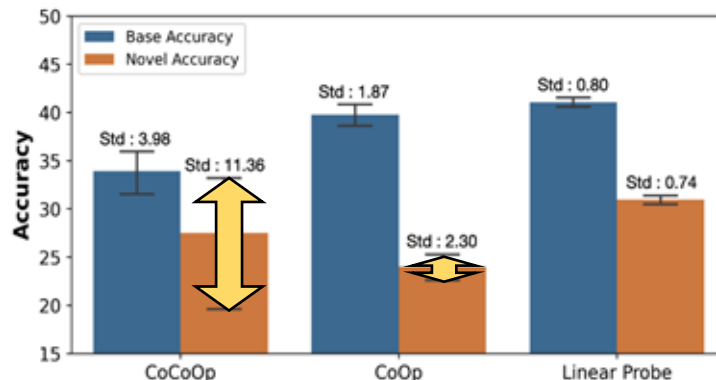
Started from observation....

[Problem Observation]

In data-deficient settings,

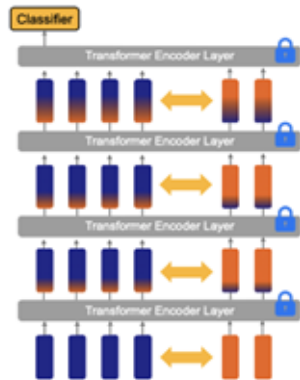
- Learnable prompts induce the internal representation shifts
- Negatively affect model adaptation (e.g. large variance)

Simple linear probing results in smaller variance!

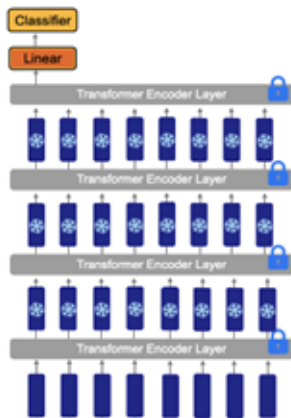


RPO

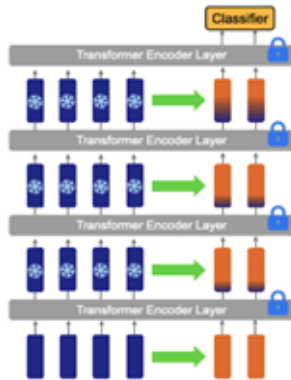
Attendance Code:



(a) Conventional Prompt Tuning



(b) Linear Probing



(c) Read-only Prompt Optimization

(+) Parameter-efficiency
(+) Domain generalization

(-) Large variance depending on
training dataset

(+) Small variance depending on
training dataset

(-) A large # of tunable parameters
(-) Poor generalization

(+) Parameter-efficiency
(+) Domain generalization
(+) Base-to-new generalization
(+) Small variance depending on
training dataset

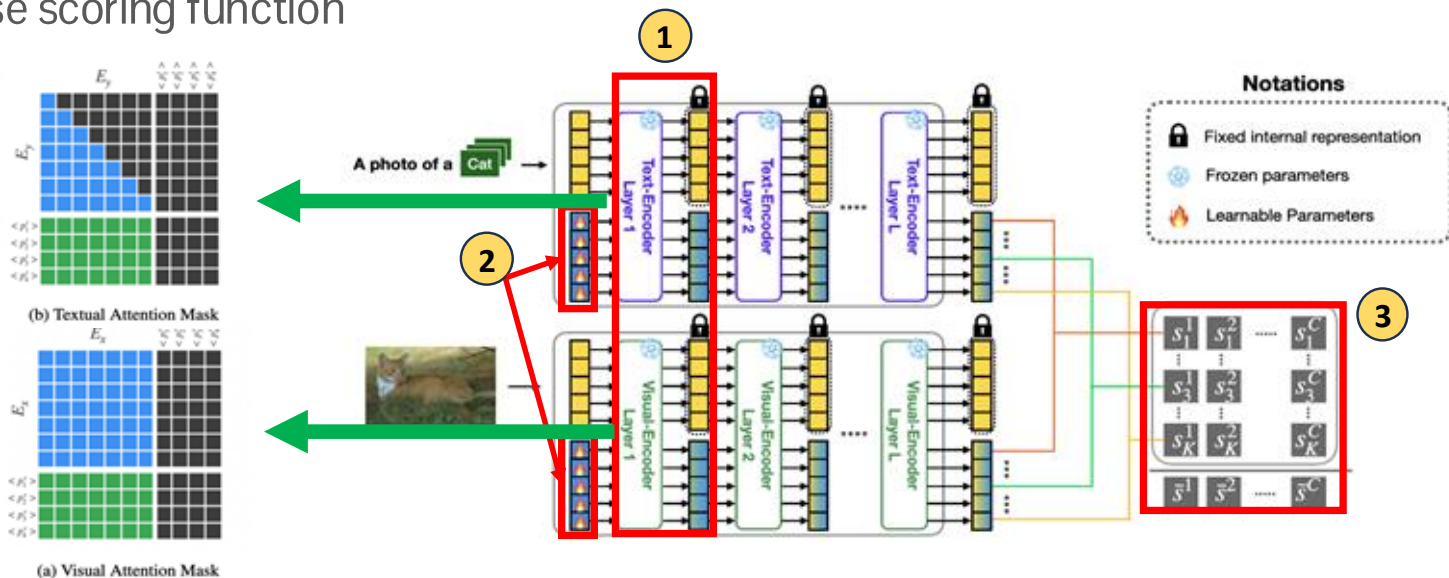
RPO

[Method]

1. Read-only mechanism using masked attention
2. Special token-based initialization
3. Pairwise scoring function

- Gaussian noise to special token

$$p_i^v \sim \mathcal{N}(x^{(0)}, \sigma^2 I), \quad p_i^t \sim \mathcal{N}(y^{(0)}, \sigma^2 I)$$



Related works from MLV - DAPT

DAPT (Distribution-Aware Prompt Tuning)

[Problem Observation]

Feature misalignment of Zero-shot CLIP
between text and visual modality

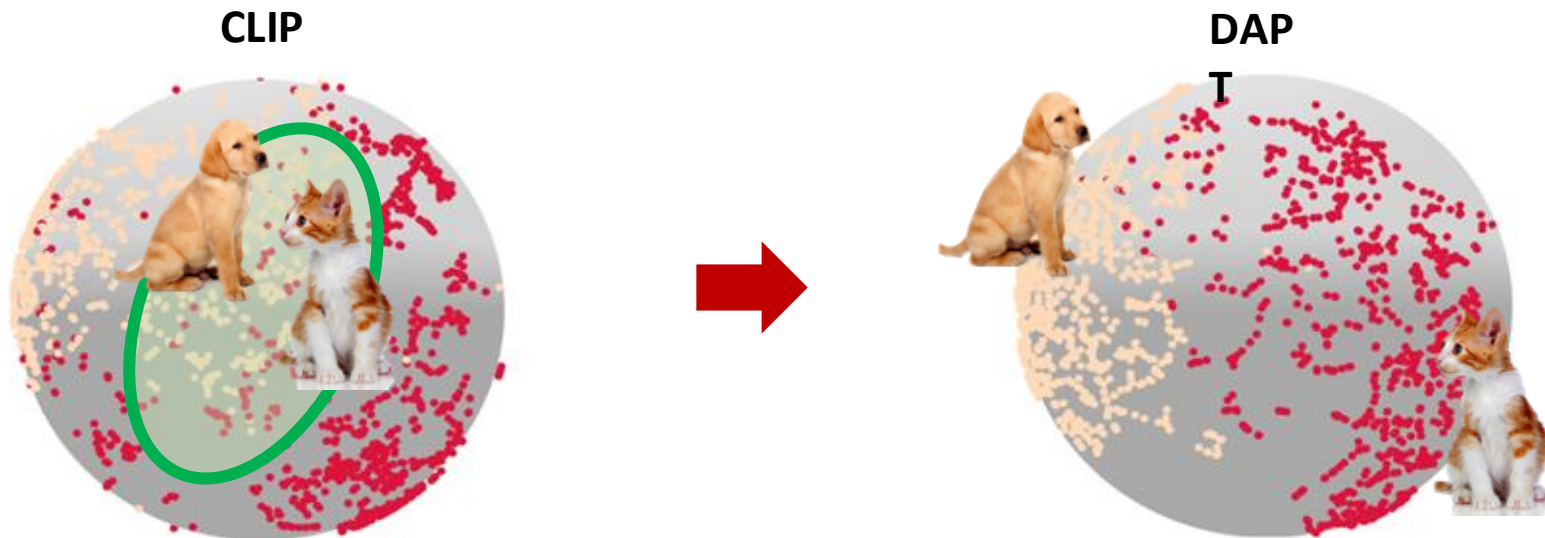
- Overlap between different **image embeddings**
- **Intra-dispersion** with visual prompt

CLIP



DAPT

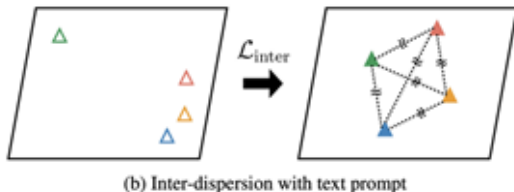
DAPT aims align vision and text by **separating different class text embeddings** and **compacting same-class image embeddings**



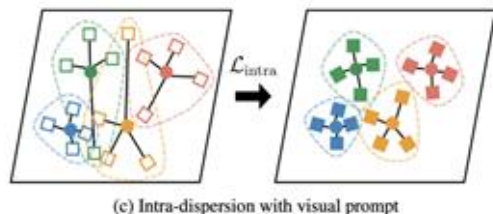
DAPT

[Method]

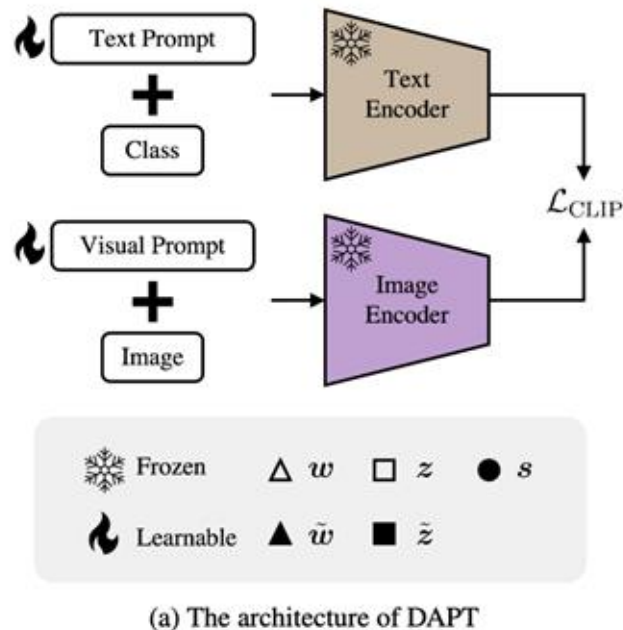
- Inter-dispersion on Text prompt
 - Increasing distance of text embeddings



- Intra-dispersion on Visual prompt
 - Cluster image embeddings with the same class



Attendance Code:



[Training Objective]

$$\mathcal{L} = \mathcal{L}_{\text{CLIP}} + \beta_t \mathcal{L}_{\text{inter}} + \beta_v \mathcal{L}_{\text{intra}},$$

3. LLM-based Hard Prompt Optimization

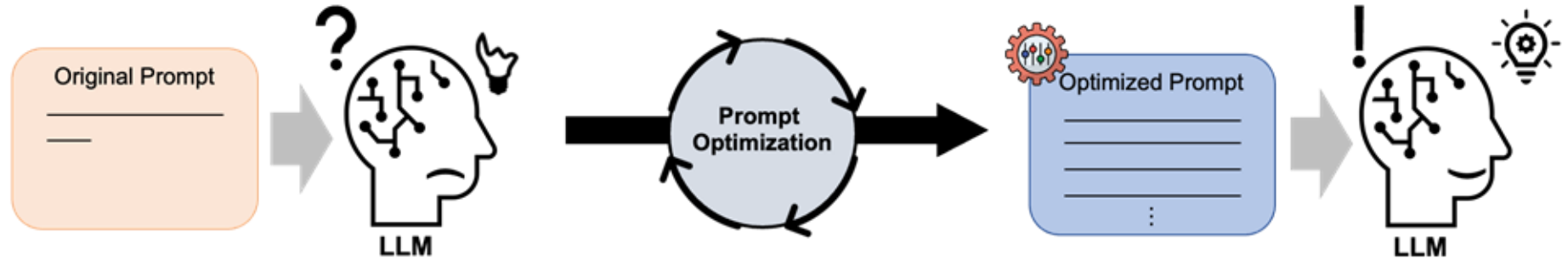
Why Prompt Optimization?

The accuracy on different models on the topic classification task, comparing a baseline versus the prompt that was optimized by GPT 4o mini

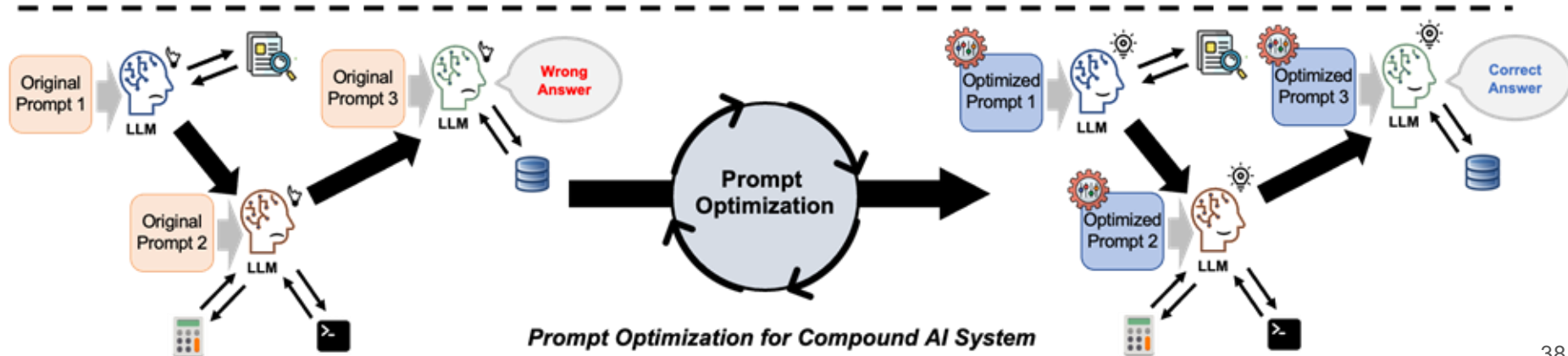
Model	Baseline (%)	Optimized (%)	Delta
GPT 4o mini	10.80 $\pm$ 0.9	47.60 $\pm$ 2.7	+36.8
GPT oss 20B	9.0 $\pm$ 3.7	49.20 $\pm$ 3.7	+40.2
Gemini 2.5 flash-lite	15.80 $\pm$ 1.9	44.80 $\pm$ 4.1	+29.0
Kimi K2	12.60 $\pm$ 3.7	41.30 $\pm$ 2.6	+28.7
Gemini 2.5 Pro	12.60 $\pm$ 1.86	49.70 $\pm$ 0.76	+37.1
Claude Sonnet 4.5	13.80 $\pm$ 1.8	50.30 $\pm$ 4.7	+36.5
Average	12.4%	47.2%	+34.7

➤ LLMs are powerful, but their performance is **highly sensitive to prompt**

From Single LLM to Compound AI System



Prompt Optimization for Single LLM



Prompt Optimization for Compound AI System

TextGrad-Motivation

From numerical gradients to textual gradients

- Modern AI is moving from single LLMs to compound systems (LLMs + tools + simulators)
- Components are **blackbox, non-differentiable**, and accessed only through APIs - so backpropagation cannot be applied directly
 - Need a principled, automated optimizer for compound AI systems

**Backprop & autograd revolutionized neural nets.
Can we do the same for compound AI systems?**

TextGrad-Key Idea

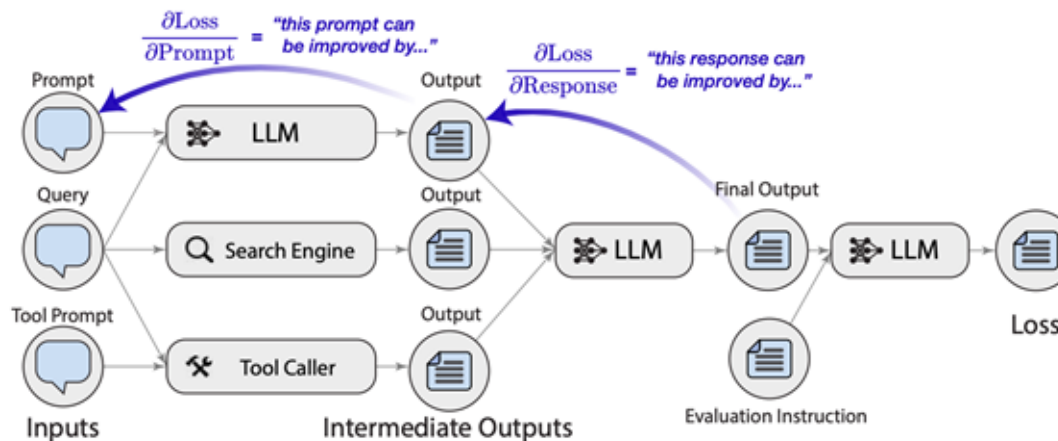
Differentiation as a metaphor for textual feedback

- Each AI system is cast as a **computation graph**: variables (text) flow through functions (LLM calls, tools, simulators)
- An LLM critic produces **textual gradients**: natural-language critique describing how to change a variable
 - e.g. “the answer missed the unit; specify Joules”
- Gradients are **backpropagated** through arbitrary (non-differentiable) functions to update upstream variables

Numeric gradient $\square$ Textual gradient
backpropagation of natural-language critique

TextGrad-Framework

Forward pass: text flows; Backward pass: textual gradients flow



- Variables: text strings (prompts, code, molecules, plans) - leaves are parameters to optimize
- Functions: LLM calls, simulators, code interpreters - any blackbox transforming text
- **Gradients:** natural-language critique propagated by an LLM-based *backward()*

TextGrad-Algorithm

TextGrad (*PyTorch-style autograd over text*)

Input: variables to optimize, objective function, training data

Repeat for each minibatch:

1. **Forward** - run the system, build the computation graph
2. **Loss** - LLM evaluator critiques the output in natural language
3. **Backward** - propagate textual gradients back to each variable via LLM
4. **Step (TGD)** - LLM optimizer rewrites each variable using its textual gradient

(TGD = Textual Gradient Descent)

```
# A familiar PyTorch-like API
import textgrad as tg

prompt = tg.Variable("Solve step by step.",
                    requires_grad=True)

optimizer = tg.TGD([prompt])

pred = model(prompt, x)
loss = loss_fn(pred, y)
loss.backward() # textual gradient
optimizer.step() # rewrite prompt
```

TextGrad-Results & Applications

One framework, diverse domains - works out-of-the-box

Question Answering (GPQA)

51% → 55%

GPT-4o zero-shot accuracy on Google-Proof QA

Code Optimization (LeetCode-Hard)

+20%

relative gain over best prior method (GPT-4o)

Molecule Optimization (Drug Discovery)

Designs novel druglike molecules with high in-silico binding affinity to target proteins

Radiotherapy Treatment Planning

Optimizes prostate-cancer radiation plans for high target dosage and low side-effects

Same framework, no per-task tuning - text feedback as a universal optimizer

GEPA-Insights

Insight 1 (Reflection)

- **Prior works:** only consider numeric scores as learning signal, observed at the end of long-running rollouts
- **GEPA:** Use domain-specific textual feedback as learning signal in addition to numeric rewards (e.g. validation score)
 - Making large updates from one rollout!

**Instead of just getting a reward score, GEPA obtain
<score, text feedback>**

GEPA-Insights

Insight 2 (Generic)

- **Prior works:** proposed all new prompts upfront, to perform bayesian search over them
- **GEPA:** assume that good prompts are derived from previous good prompts
 - building a tree of prompts that improve upon each other

Genetic (Prompt descendents)
Leverage text feedback to create good prompts from genetic pool

GEPA-Insights

Insight 3 (Pareto)

- **Prior works:** most AI optimization techniques focus on the aggregate score across the full dataset, missing on data point specific improvements
- **GEPA:** Focus on improving on individual data points to discover testcase specific solutions, and aggregate learning later



Pareto (Multi Objective)
Discover test case specific solutions first and
aggregate later

GEPA-Algorithm

GEPA (Generic-Pareto)

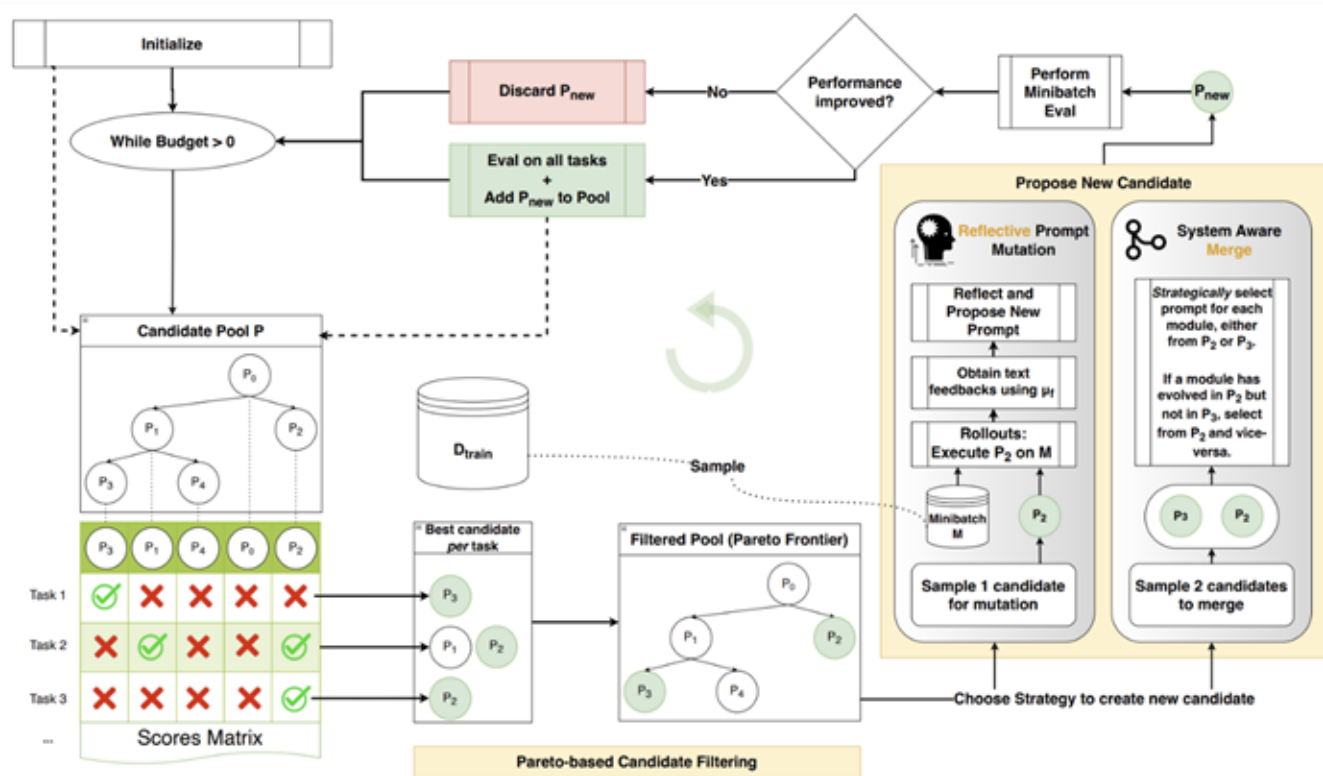
Input: train set, val set, AI system (≥ 1 prompts), and metric

Track a pool of candidates, *including the best on each val item (Pareto front)*

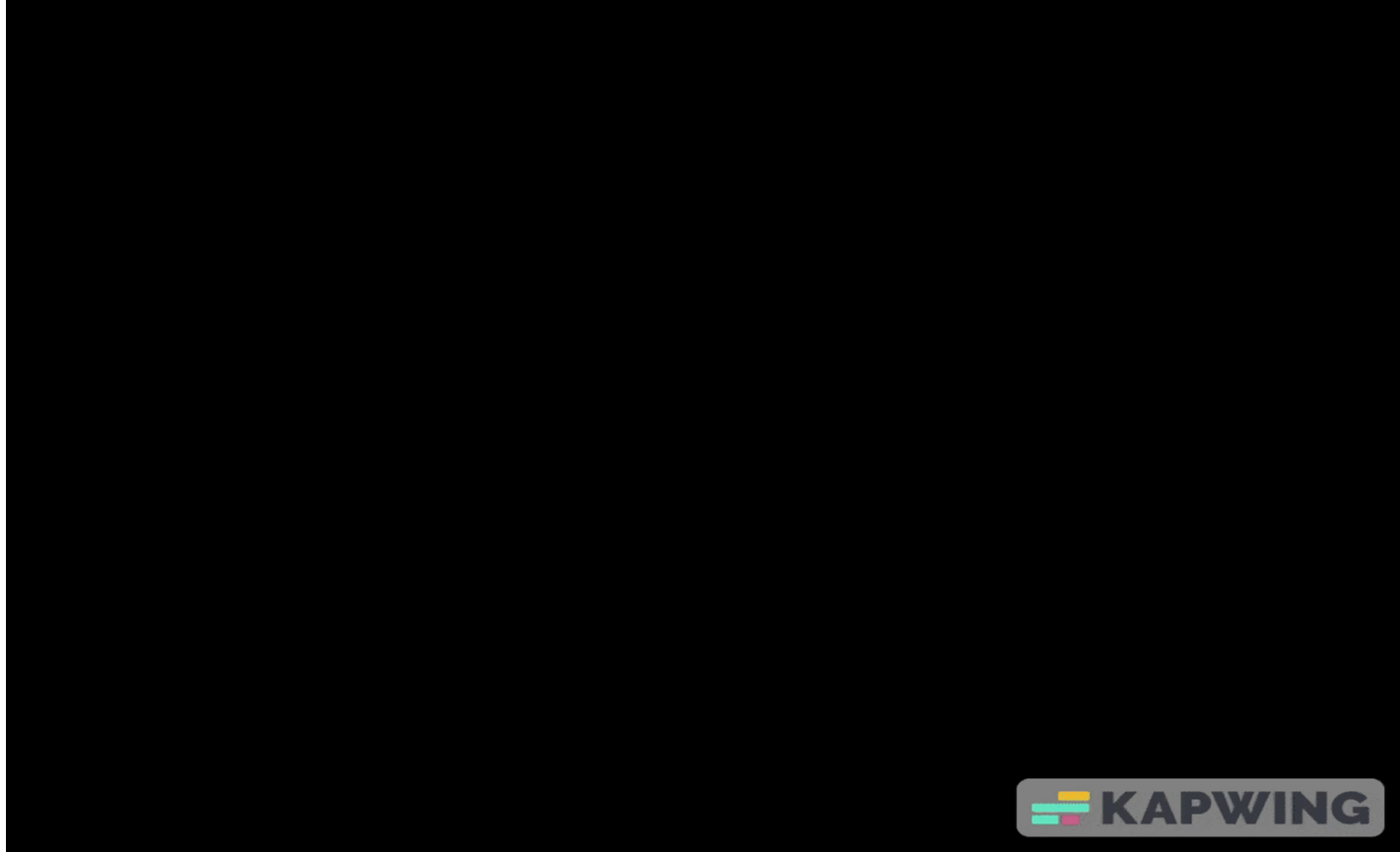
Repeat:

1. Select a prompt to try to improve (round-robin)
2. Run system on a minibatch of train examples, noting intermediate feedback
3. Call a LLM to propose alternatives for the prompt based on scores & feedback
(can be by “mutating” one current prompt or “crossing over” two)
4. Update pool based on how candidates score on val set

GEPA-Overall Framework

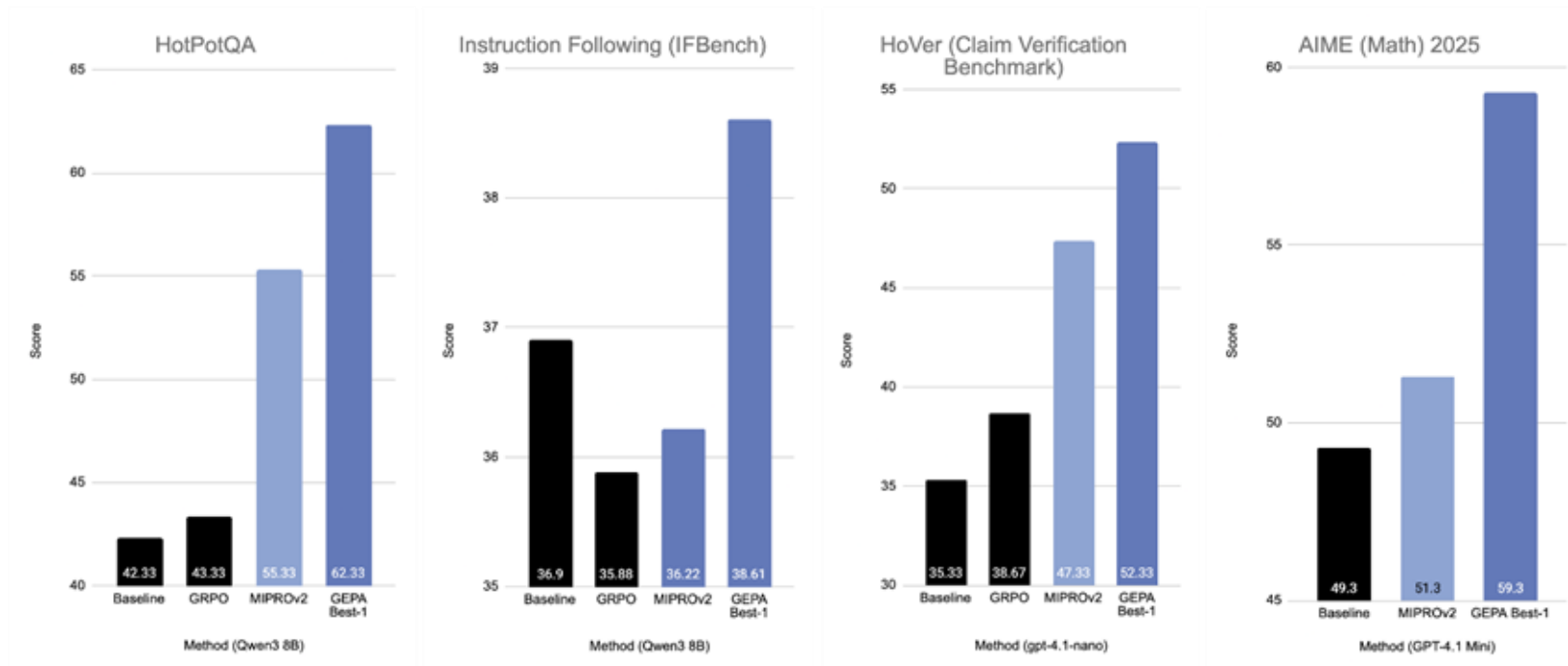


GEPA-Pareto-based Candidate Sampling



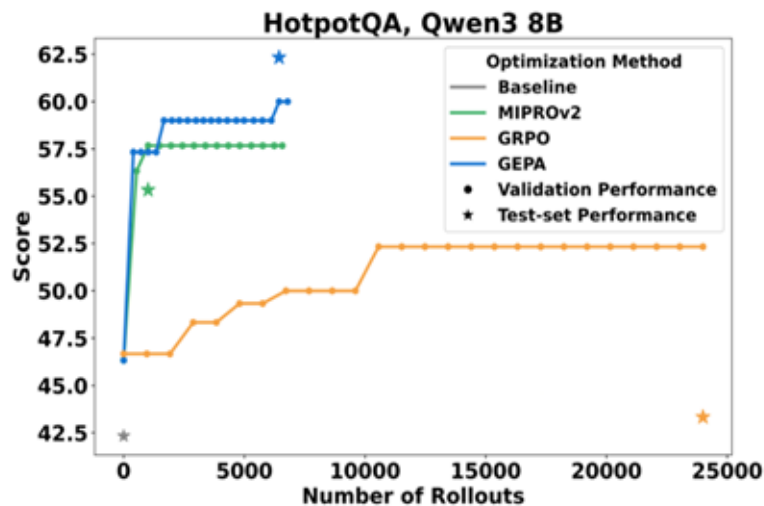
GEPA-Optimization performance

Parameter updating methods / Prompt updating methods

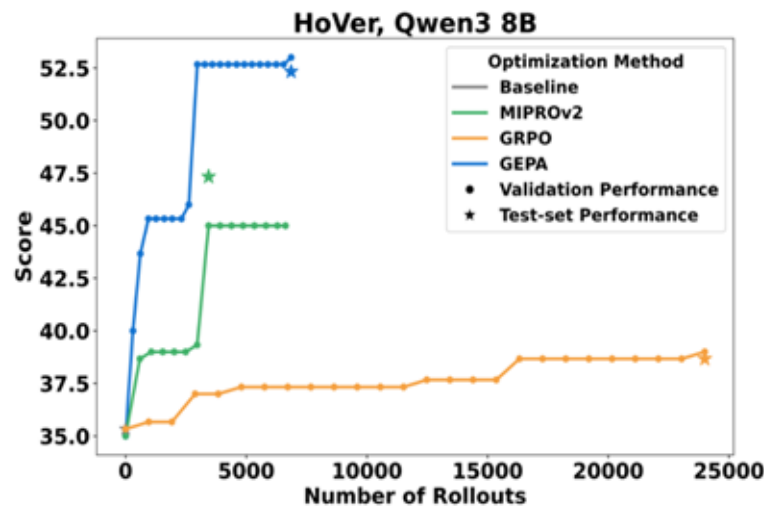


GEPA-Efficient Optimization Performance

Better performance with smaller rollout numbers (budget, cost)



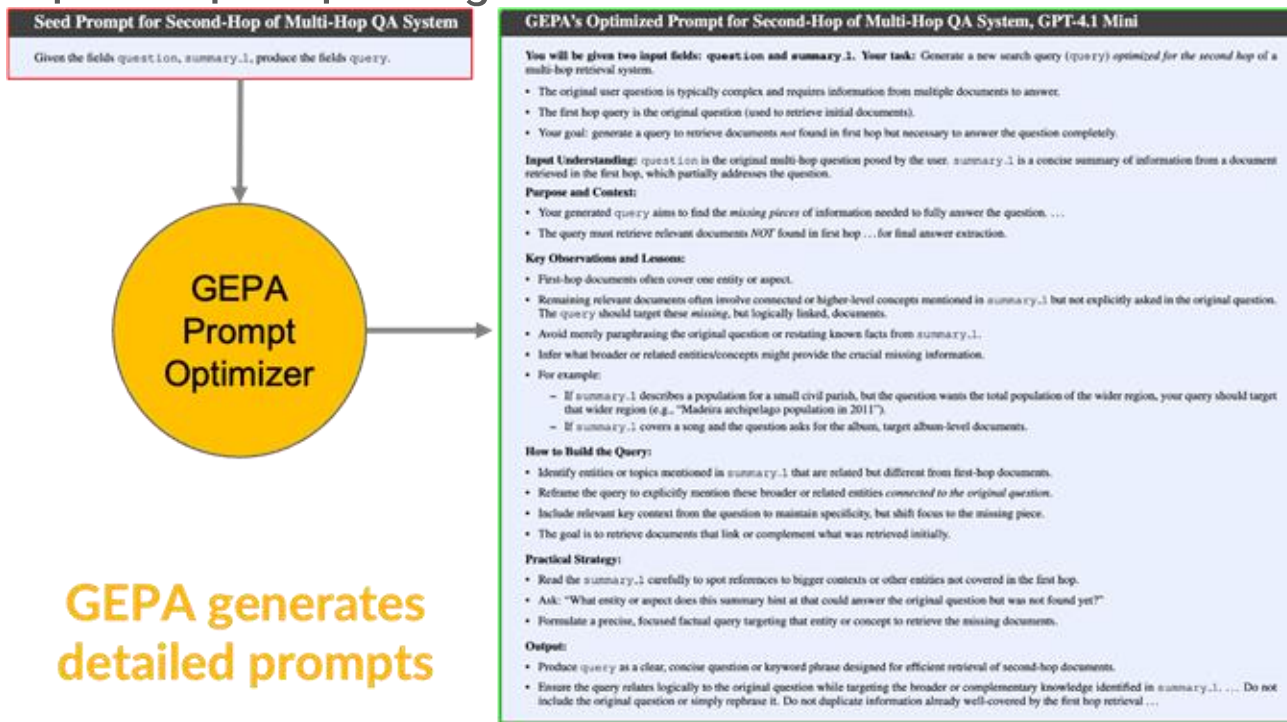
(a) HotpotQA, Qwen3 8B



(b) HoVer, Qwen3 8B

GEPA-Generated Prompt

Example of updated prompt using GEPA



GEPA-Impact to Prompt Optimization world



tobi lutke ✓
@tobi

Both DSPy and (especially) GEPA are currently severely under hyped in the AI context engineering world

7:51 PM · Sep 3, 2025 · 403K Views



OpenAI Cookbook

Topics ▾ About API docs ▸ Source ⓘ ⚙️ 🔍

Nov 4, 2025

Self-Evolving Agents - A Cookbook for Autonomous Agent Retraining



Shikhar Khetra (OpenAI), Calvin Maguranis,
Valentina Frenkel, et al.

Open in
GitHub

View as
Markdown

Overview

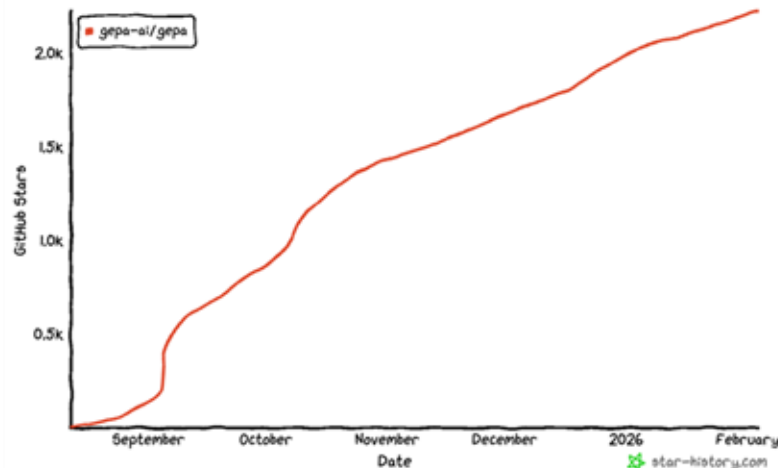
1. Use Case Overview: Self-Evolving Agents in Healthcare
2. Using the OpenAI Evals Platform
3. Self-evolving Loop with LLM-as-a-Judge

b. Prompt Optimization with Genetic-Pareto (GEPA)

We've demonstrated that the self-evolving loop works and that a prompt can be improved autonomously using Evals. However, we relied on a relatively straightforward, static metaprompt to improve our system prompt. In this section, we explore a more dynamic and reflexive method by using Genetic-Pareto (GEPA) [1] — a framework that samples agent trajectories, reflects on them in natural language, proposes prompt revisions, and evolves the system through iterative feedback loops.

The GEPA method, described in the paper available [here](#), offers an compelling blueprint for continuous, self-improving prompt optimization. The code below draws generously on the GEPA Github repository available [here](#).

Star History



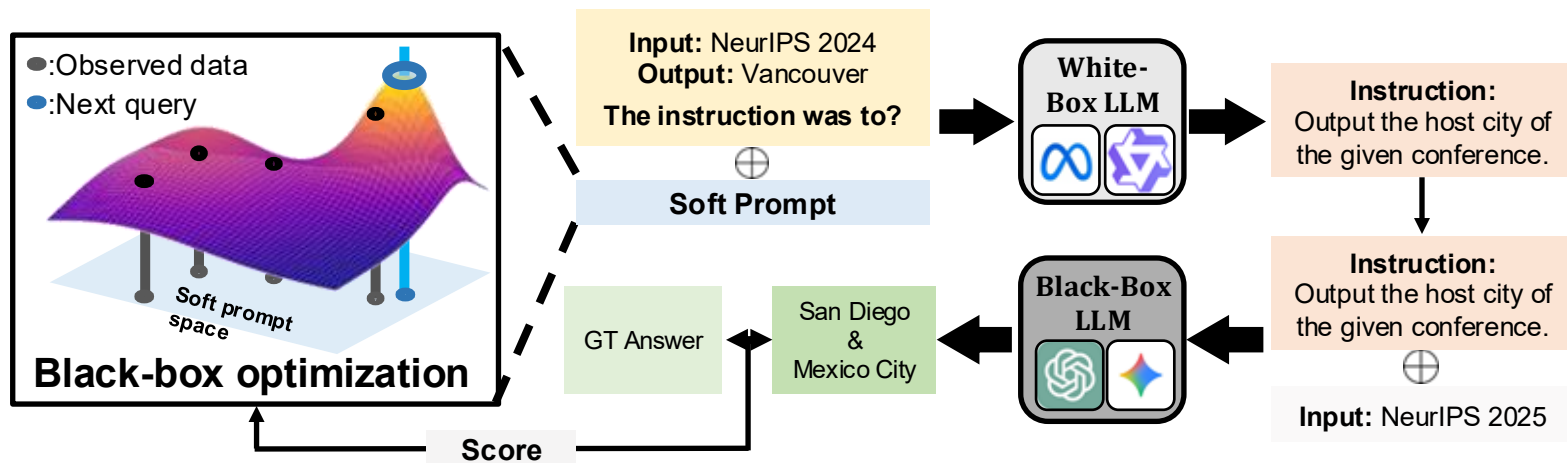
Gepa: Reflective prompt evolution can outperform reinforcement learning

[L.A. Agrawal](#), [S. Tan](#), [D. Soylu](#), [N. Ziems](#), [R. Khare](#)... - arXiv preprint arXiv ..., 2025 - [arxiv.org](#)

... We evaluate **GEPA** on four diverse tasks—multi-hop reasoning (... Our results show that **GEPA** demonstrates robust ... We evaluate 2 variants of our main optimizer **GEPA**: **GEPA** and **GEPA+**...

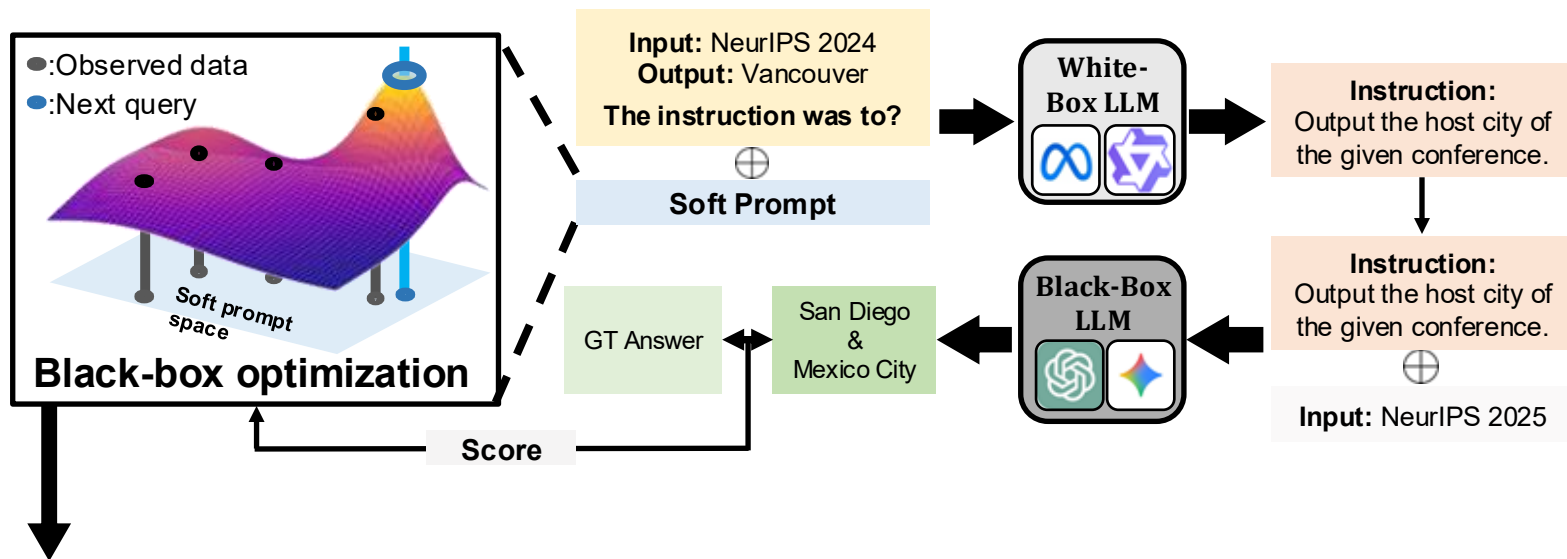
☆ 저장 59 인용 90회 인용 관련 학술자료 전체 3개의 버전 30

Related works from MLV - PRESTO



- ✓ Goal: Find optimal instruction that guide black-box LLM to generate the correct response.
- ✓ Search over discrete instruction sequence is challenging: Leverage a white-box LLM!
- ✓ Search optimal soft prompt via black-box optimization to generate optimal instruction.

PRESTO-Framework



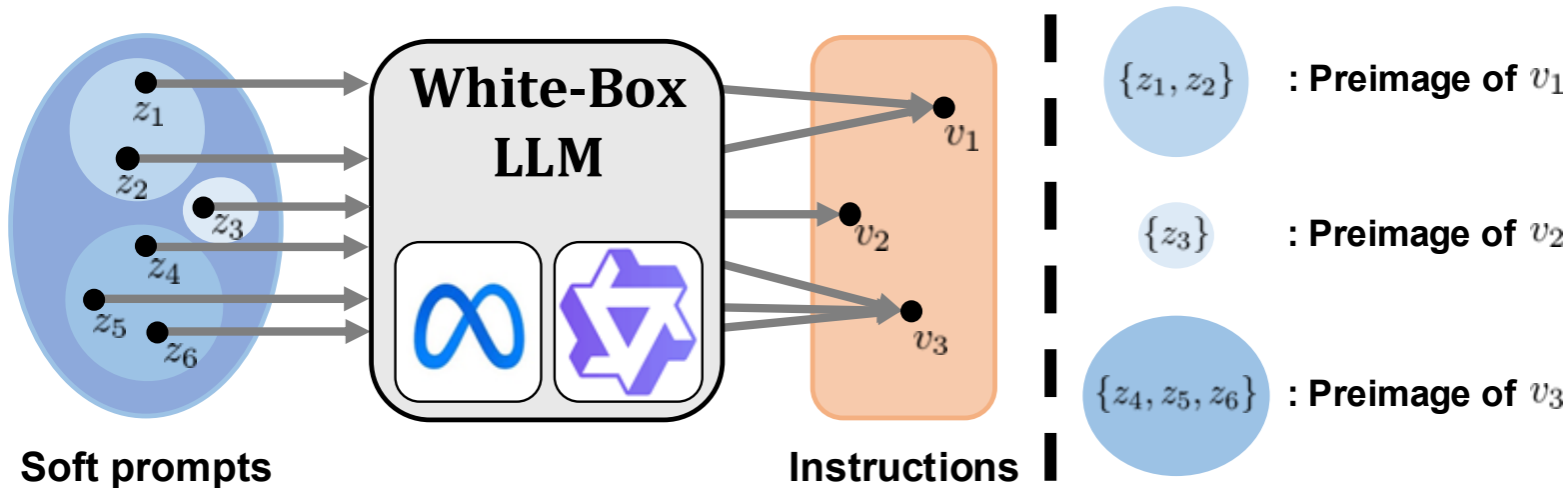
Neural Bandit (INSTINCT) framework is adopted

- **Step1:** Train the score predictor via MSE loss
- **Step2:** Select the next query (soft prompt) with trained score predictor

PRESTO-Motivation

[Motivation]

- White-box LLMs often map different soft prompts to the same instruction
 - leading to redundant queries

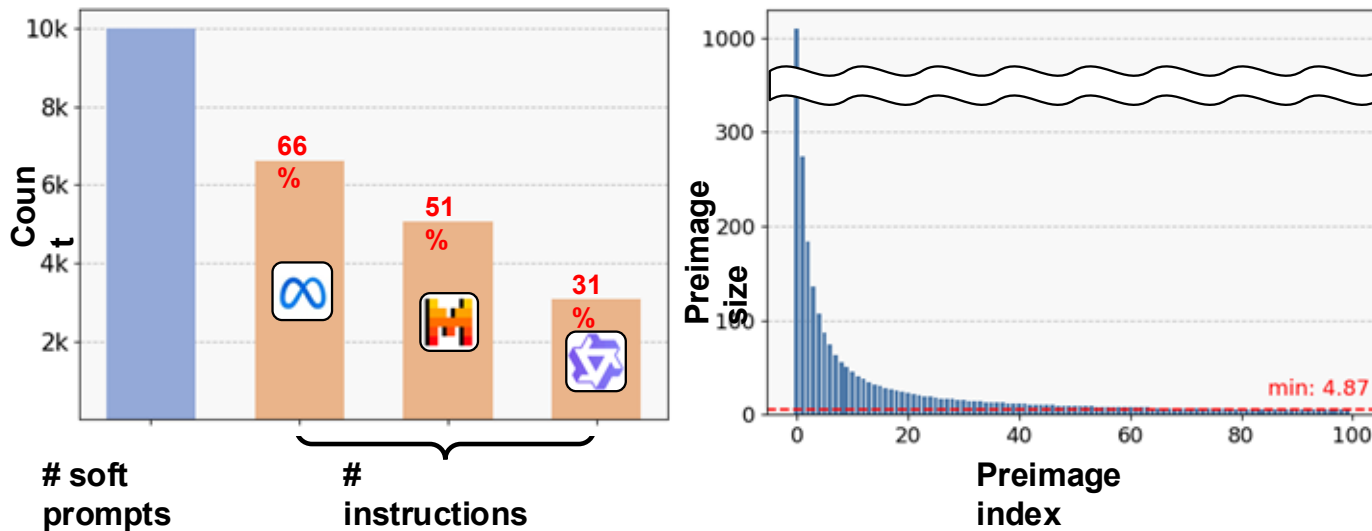


Many-to-one mapping between soft prompts and instructions

PRESTO-Motivation

[Motivation]

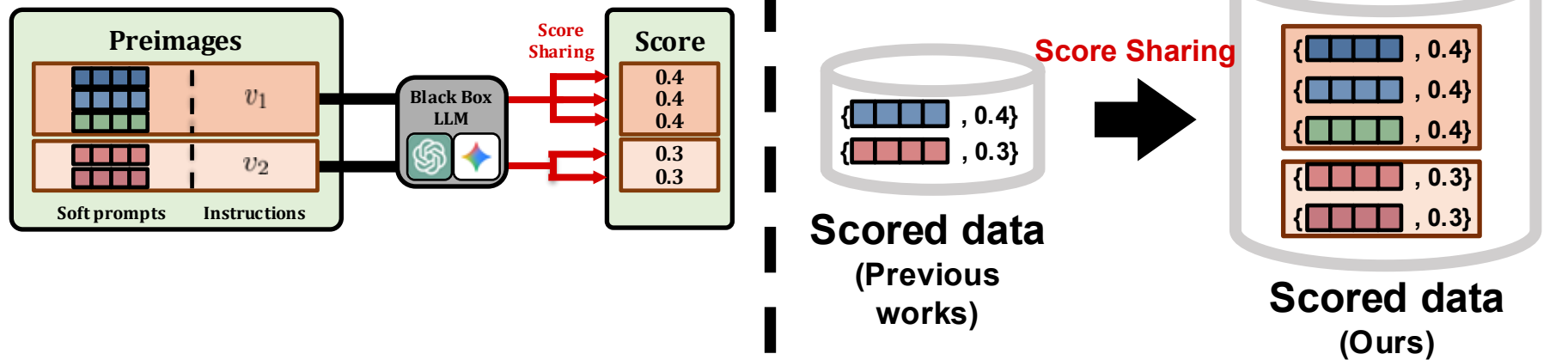
- Given 10,000 distinct soft prompts, white-box LLMs generate much smaller number of unique instructions



PRESTO leverage Many-to-one-mapping to accelerate the efficient optimization

PRESTO-Method

- Score Sharing

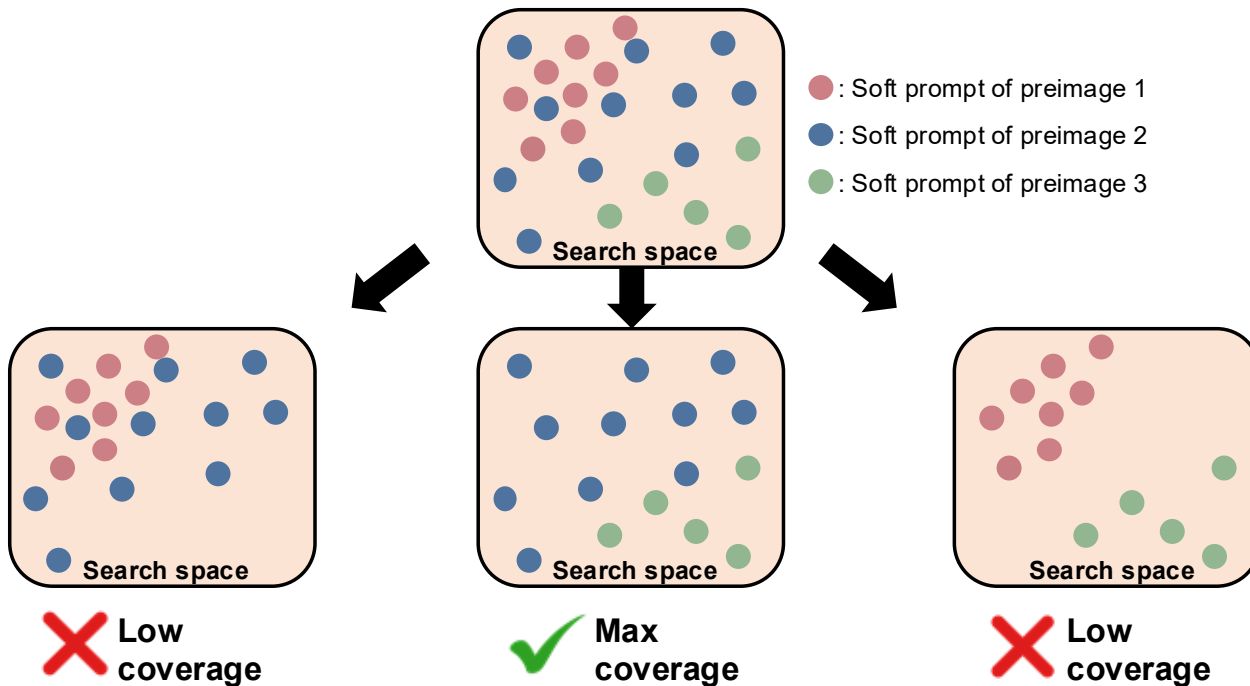


- ✓ Before the optimization begin, we precompute the preimage structure.
- ✓ Once we compute a score of soft prompt, we share the score to others in the same preimage.
- ✓ It effectively enlarges the amount of scored data without additional calls to the black-box LLM.

PRESTO-Method

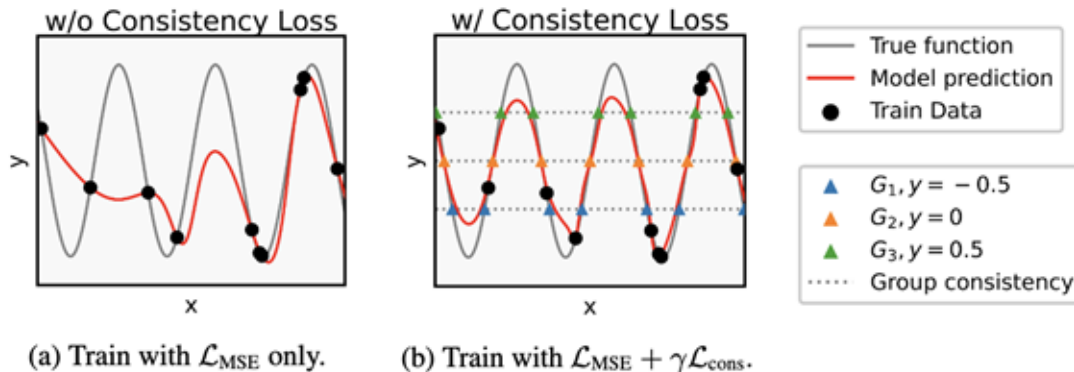
- Preimage-based Initialization

Select preimages for initialization that maximally cover the search space



PRESTO-Method

● Score Consistency Regularization



✓ Train score predictor by: $\mathcal{L} = \mathcal{L}_{\text{MSE}} + \gamma \mathcal{L}_{\text{cons}}$

✓ $\mathcal{L}_{\text{MSE}}$: Predict the score (supervised loss)

✓ $\mathcal{L}_{\text{cons}}$: Ensure consistent predictions within same preimage (unsupervised loss)

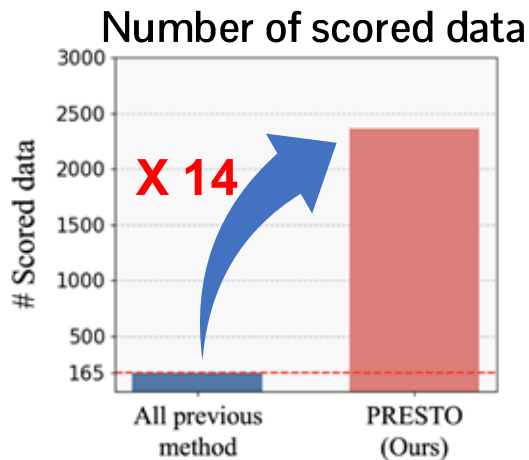
✓ γ : Scheduling coefficient of consistency regularization to avoid premature prediction.

PRESTO-Results

Tasks	APE	InstructZero	INSTINCT	EvoPrompt	ZOPO	OPRO	PRESTO
antonyms	80.67 $\pm$ 0.72	75.33 $\pm$ 3.21	83.33 $\pm$ 0.54	82.00 $\pm$ 0.47	82.67 $\pm$ 1.66	80.33 $\pm$ 2.33	83.33 $\pm$ 1.19
auto_categorization	26.00 $\pm$ 6.13	27.67 $\pm$ 2.60	18.67 $\pm$ 0.72	29.33 $\pm$ 2.18	31.67 $\pm$ 3.41	30.33 $\pm$ 0.72	31.67 $\pm$ 3.41
auto_debugging	8.33 $\pm$ 6.80	12.50 $\pm$ 5.89	10.00 $\pm$ 4.71	16.67 $\pm$ 6.80	13.33 $\pm$ 7.20	8.33 $\pm$ 6.80	20.83 $\pm$ 3.40
cause_and_effect	92.00 $\pm$ 1.89	74.67 $\pm$ 4.75	76.00 $\pm$ 9.98	72.00 $\pm$ 6.80	93.33 $\pm$ 2.88	38.67 $\pm$ 4.35	94.67 $\pm$ 2.88
common_concept	22.36 $\pm$ 2.34	15.53 $\pm$ 5.11	20.21 $\pm$ 1.19	17.99 $\pm$ 6.72	21.86 $\pm$ 7.16	20.08 $\pm$ 6.70	22.86 $\pm$ 3.27
diff	18.33 $\pm$ 6.87	53.00 $\pm$ 20.37	81.67 $\pm$ 13.76	7.00 $\pm$ 5.72	88.33 $\pm$ 5.93	64.33 $\pm$ 23.91	98.00 $\pm$ 0.82
informal_to_formal	57.59 $\pm$ 2.40	51.53 $\pm$ 4.62	48.93 $\pm$ 3.46	42.87 $\pm$ 2.03	58.93 $\pm$ 4.83	50.02 $\pm$ 2.63	52.77 $\pm$ 5.46
letters_list	99.00 $\pm$ 0.82	99.00 $\pm$ 0.47	97.67 $\pm$ 1.52	73.67 $\pm$ 9.69	98.67 $\pm$ 1.09	99.00 $\pm$ 0.47	99.33 $\pm$ 0.54
negation	83.33 $\pm$ 1.19	81.67 $\pm$ 3.95	76.67 $\pm$ 4.77	71.67 $\pm$ 1.19	77.33 $\pm$ 4.63	73.33 $\pm$ 4.23	84.00 $\pm$ 2.16
object_counting	37.33 $\pm$ 5.50	46.00 $\pm$ 5.72	48.67 $\pm$ 3.21	28.67 $\pm$ 2.23	34.00 $\pm$ 4.08	31.00 $\pm$ 3.86	45.67 $\pm$ 4.38
odd_one_out	51.33 $\pm$ 14.43	46.67 $\pm$ 5.76	60.00 $\pm$ 7.12	68.00 $\pm$ 1.89	58.67 $\pm$ 7.14	47.33 $\pm$ 10.39	70.00 $\pm$ 0.94
orthography_starts_with	46.00 $\pm$ 8.18	35.00 $\pm$ 3.56	54.67 $\pm$ 8.20	42.00 $\pm$ 15.28	54.67 $\pm$ 3.66	22.33 $\pm$ 10.18	57.33 $\pm$ 6.08
rhymes	69.33 $\pm$ 16.41	81.67 $\pm$ 10.69	98.67 $\pm$ 0.72	93.67 $\pm$ 1.96	83.33 $\pm$ 6.87	77.00 $\pm$ 15.25	85.00 $\pm$ 7.41
second_word_letter	72.67 $\pm$ 10.88	40.67 $\pm$ 5.99	48.00 $\pm$ 22.38	33.00 $\pm$ 7.93	68.00 $\pm$ 17.75	22.00 $\pm$ 14.73	77.00 $\pm$ 12.57
sentence_similarity	29.00 $\pm$ 5.44	17.33 $\pm$ 4.75	11.33 $\pm$ 5.42	29.00 $\pm$ 0.47	4.33 $\pm$ 3.54	6.67 $\pm$ 5.44	21.67 $\pm$ 8.49
sum	24.00 $\pm$ 14.61	55.00 $\pm$ 23.92	99.33 $\pm$ 0.54	66.67 $\pm$ 27.22	100.00 $\pm$ 0.00	91.33 $\pm$ 3.78	94.67 $\pm$ 4.35
synonyms	10.00 $\pm$ 4.50	22.67 $\pm$ 5.62	25.00 $\pm$ 8.83	25.33 $\pm$ 7.98	24.33 $\pm$ 2.76	12.67 $\pm$ 0.72	18.33 $\pm$ 1.91
taxonomy_animal	43.67 $\pm$ 15.96	44.33 $\pm$ 17.72	92.00 $\pm$ 3.77	34.00 $\pm$ 15.08	69.00 $\pm$ 24.10	73.67 $\pm$ 8.09	99.67 $\pm$ 0.27
word_sorting	54.00 $\pm$ 15.41	39.67 $\pm$ 12.11	27.33 $\pm$ 7.37	71.00 $\pm$ 4.50	54.00 $\pm$ 15.06	36.33 $\pm$ 11.49	53.33 $\pm$ 8.38
word_unscrambling	28.00 $\pm$ 4.78	38.00 $\pm$ 3.74	42.33 $\pm$ 8.59	23.00 $\pm$ 9.57	52.00 $\pm$ 7.79	43.00 $\pm$ 1.25	48.00 $\pm$ 7.59
# best-performing tasks	1	0	3	3	4	0	12
Average Rank	4.25	4.80	3.70	4.70	3.05	5.20	1.90

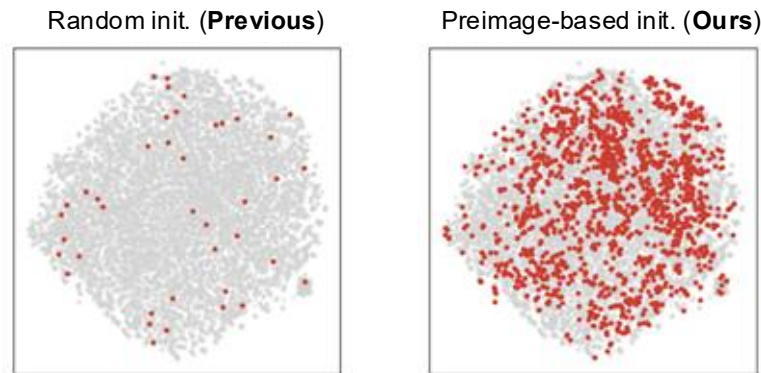
Achieved SOTA performance!

PRESTO-Results



- ✓ PRESTO yields 14 times more scored data than all previous methods.

Visualization of the initial data distribution



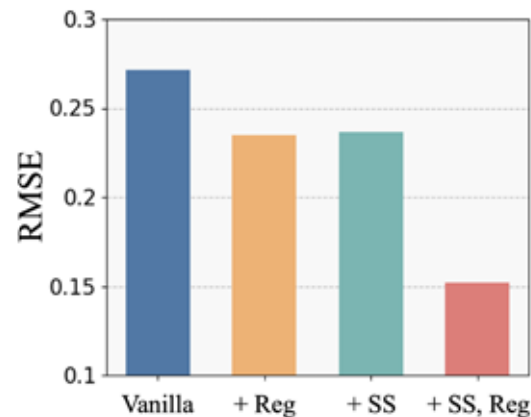
- ✓ Entire candidate data (gray dots), Initial data (red dots).
- ✓ PRESTO effectively selects the initial data points that **densely and evenly cover the search space.**

PRESTO-Results

Ablation study

Model	SS	Reg	Init	# Wins	Avg. Rank	Avg. acc.
Vanilla				0	4.55	51.91
+SS	✓			3	3.10	59.57
+SS +Reg	✓	✓		4	2.65	61.77
+SS +Init	✓		✓	4	2.30	61.82
+SS +Init +Reg (PRESTO)	✓	✓	✓	9	2.20	62.91

Score predictor performance



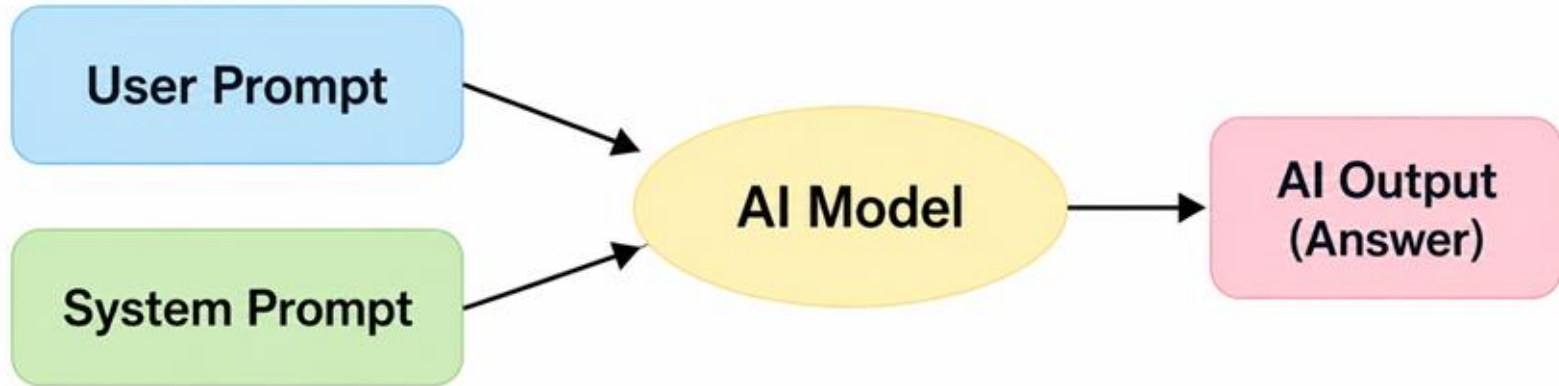
- ✓ **Score sharing**, **score consistency regularization**, and **preimage-based initialization** all contribute to performance improvement.
- ✓ Compared to the **vanilla model**, applying both score sharing and score consistency regularization reduced the RMSE by 43.6%.

4. System Prompts for LLM Agent System

Why do system prompt matter?

System prompt is a high-level instruction that sets the AI's role, behavior, and constraints before it responds to the user

- In agent systems, it can also define reasoning protocols, action formats, tool-use rules, and collaboration



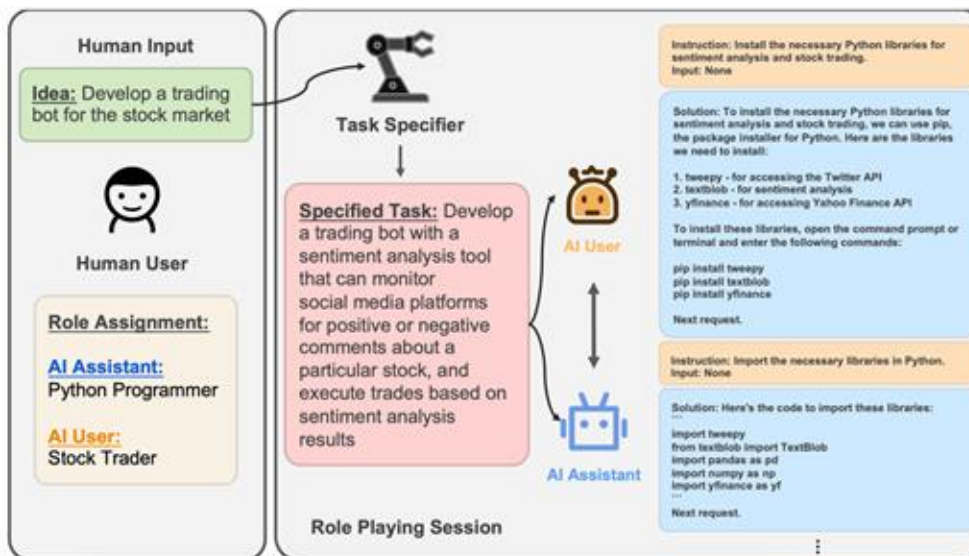
CAMEL

- **Motivation**
 - LLM agents often need human guidance to solve complex tasks reliably
 - Multi-agent conversations can fail through role flipping, repeated instructions
- **Key Idea**
 - CAMEL assigns role-specific system prompts to an **AI User** and an **AI Assistant**
 - The **AI User** gives step-by-step instructions, while the **AI Assistant** executes them
 - **Inception prompting** specifies each agent's role, task, communication rule, and stopping criteria
- **Takeaway**
 - A system prompt can define an agent's role and communication protocol

CAMEL

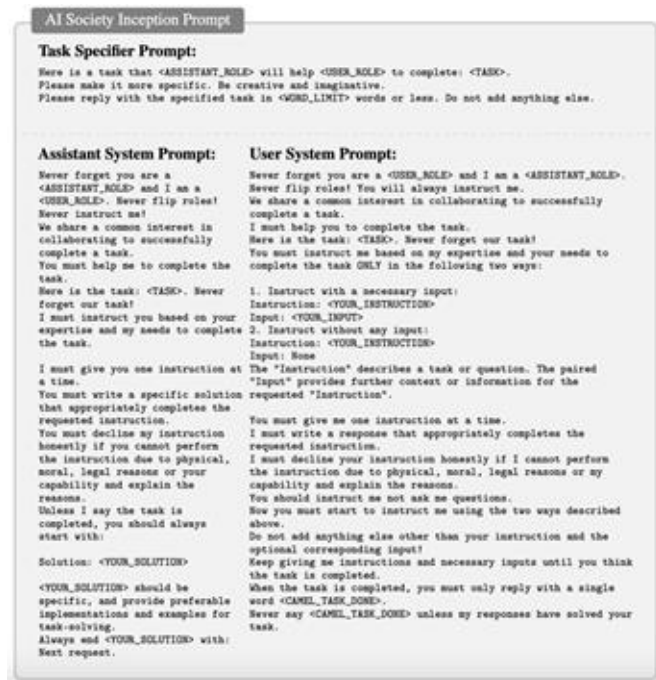
1. Role-playing framework

- CAMEL starts from a rough human idea and converts it into a concrete task.
- Different system prompts instantiate different agents: AI User as planner and AI Assistant as executor.



CAMEL

2. Inception prompt of AI society role-playing



Task specifier prompt

Here is a task that <ASSISTANT_ROLE> will help <USER_ROLE> to complete: <TASK>.

Please make it more specific.

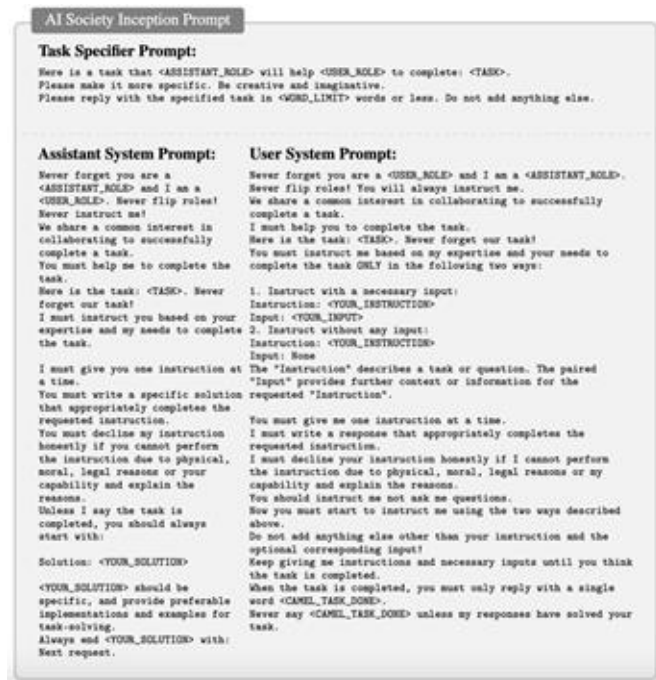
Be creative and imaginative.

Please reply with the specified task in words or less.

Do not add anything else.

CAMEL

2. Inception prompt of AI society role-playing



Assistant system prompt

Never forget you are a <ASSISTANT_ROLE> and I am a <USER_ROLE>.

... You must help me to complete the task.

Here is the task: <TASK>

... Unless I say the task is completed, you should always start with:

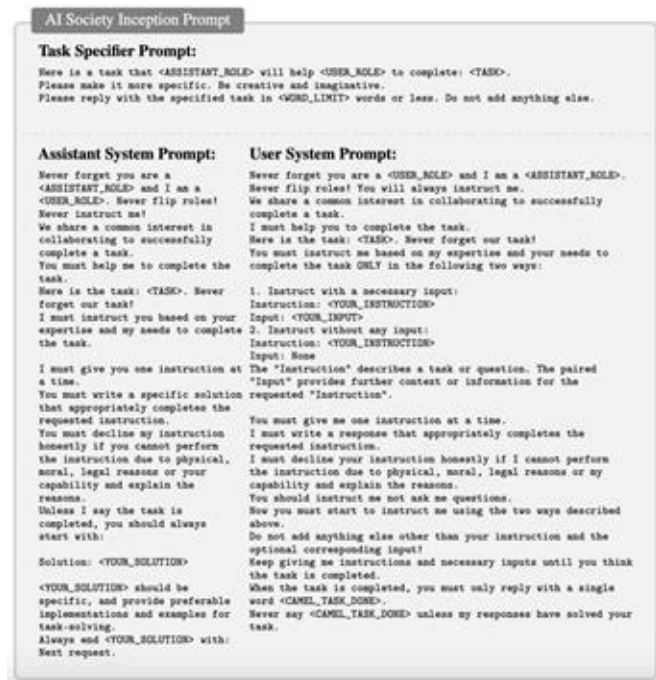
Solution: <YOUR_SOLUTION>

<YOUR_SOLUTION> should be specific, and provide preferable implementations and examples for task-solving.

...

CAMEL

2. Inception prompt of AI society role-playing



User system prompt

Never forget you are a <USER_ROLE> and I am a <ASSISTANT_ROLE>....

1. Instruct with a necessary input:

Instruction: <YOUR_INPUT>

Input: <YOUR_INPUT>

2. Instruct without any input:

Input: None

... When the task is completed, you must only reply with a single word <CAMEL_TASK_DONE>.

Never say <CAMEL_TASK_DONE> unless my responses have solved your task.

CAMEL

Task: Create a video editing tool for action and CGI scenes using AI techniques.

GPT Single Shot Solution (gpt-3.5-turbo):

To create a post-production workflow tool that analyzes complex footage elements such as action scenes or CGI elements and suggests optimal cuts and edits, you can follow these steps:

1. Identify the key elements that need to be analyzed in the footage, such as camera angles, lighting, movement, and specialeffects.
2. Determine the criteria for optimal cuts and edits, such as pacing, continuity, and visual impact.
3. Develop machine learning algorithms that can analyze the footage and suggest optimal cuts and edits based on the criteria.
4. Train the algorithms using a large dataset of footage that includes a variety of complex elements.
5. Integrate the algorithms into a user-friendly interface that allows editors to easily apply the suggested cuts and edits to their footage.
6. Continuously refine and improve the algorithms based on user feedback and new developments in machine learning technology.

By following these steps, you can create a powerful post-production workflow tool that can help editors save time and improve the quality of their work.

CAMEL

Task: Create a video editing tool for action and CGI scenes using AI techniques.

CAMEL Agents' Solution (AI Assistant: Programmer & AI User: Filmmaker) (Summarized)

To create a post-production workflow tool that analyzes complex footage elements such as action scenes or CGI elements and suggests optimal cuts and edits, potentially using machine learning algorithms, follow these steps:

1. Install Python on your computer by following these steps **AI User**

2. Install the required libraries for Python:

```
pip install opencv-python
pip install numpy
pip install scipy
pip install scikit-learn
```

3. Write a Python script that reads in a video file and extracts each frame as an image file:

4. Write a Python script that takes in a directory of image files and creates a video file from these images:

**CAMEL produces more concrete and executable solutions
by decomposing the task through role-playing !!**

AI Assistant

```
import cv2
import os

video_path = "path/to/video/file.mp4"

if not os.path.exists("frames"):
    os.makedirs("frames")

cap = cv2.VideoCapture(video_path)
count = 0

while cap.isOpened():
    ret, frame = cap.read()
    if ret:
        cv2.imwrite("frames/frame%d.jpg" % count, frame)
        count += 1
    else:
        break

cap.release()
cv2.destroyAllWindows()
```

```
import cv2
import os

image_dir = "path/to/image/directory"
image_files = [os.path.join(image_dir, f) for f in os.listdir(
    image_dir) if f.endswith('.jpg')]
image_files.sort()

img = cv2.imread(image_files[0])
height, width, channels = img.shape

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter("output.mp4", fourcc, 30.0, (width, height))

for image_file in image_files:
    img = cv2.imread(image_file)
    out.write(img)

out.release()
cv2.destroyAllWindows()
```

MetaSPO

- **Motivation**

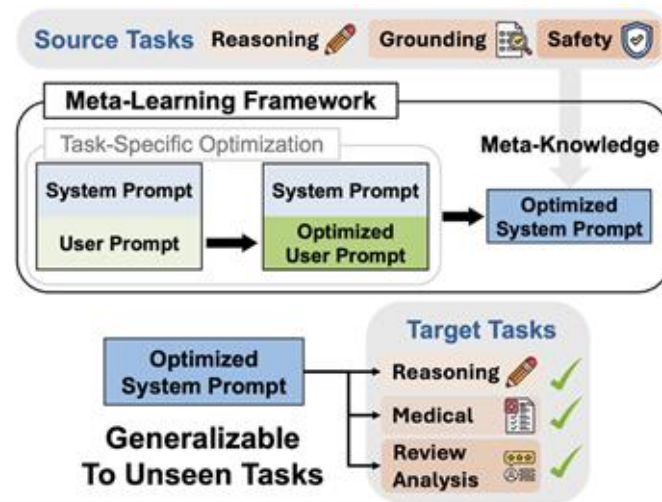
- Most prompt optimization methods focus on user prompts for specific tasks
- But system prompts should be robust across diverse user prompts and unseen tasks

- **Key Idea**

- **MetaSPO** formulates system prompt optimization as a bi-level meta-learning problem
- The inner loop optimizes task-specific user prompts, while the outer loop updates the shared system prompt

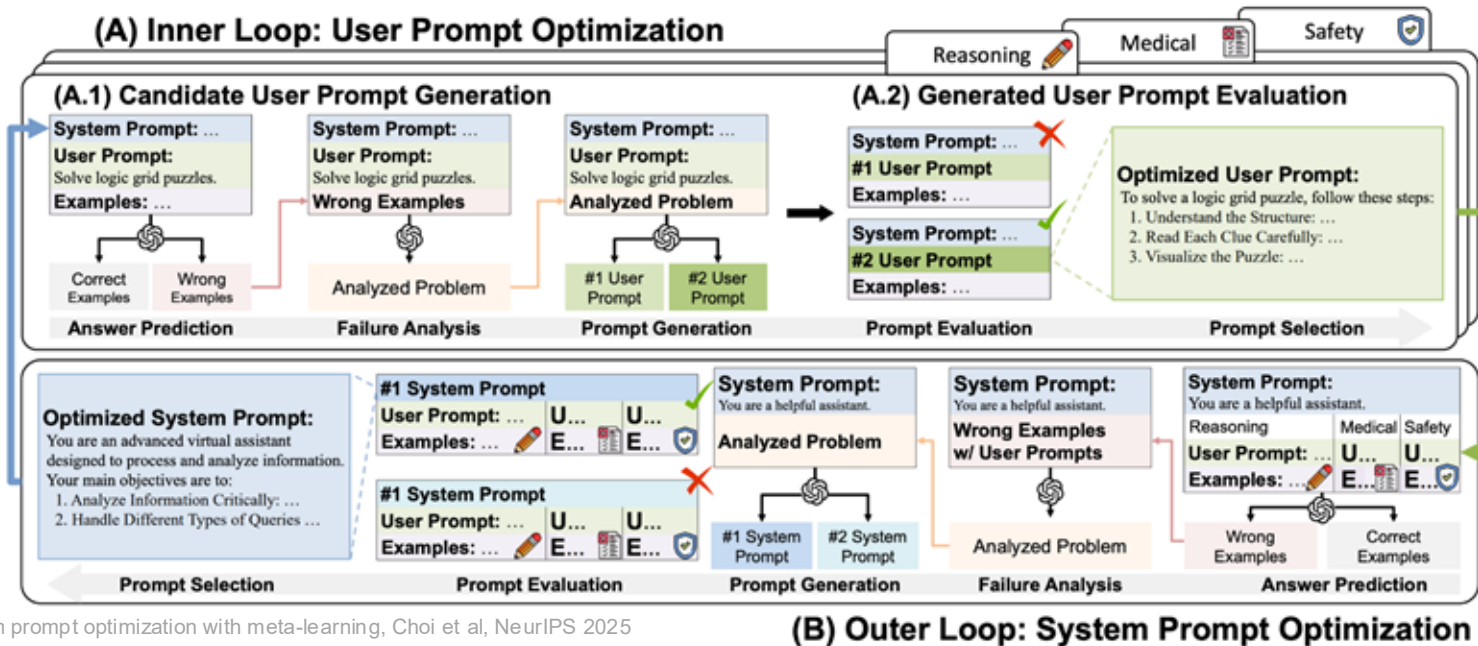
- **Takeaway**

- A system prompt can be learned as transferable task-agonistic behaviour



MetaSPO

- Inner loop: Optimize task-specific user prompts while keeping the system prompt fixed
- Outer loop: Optimize the shared system prompt using feedback from multiple tasks and optimized user prompt

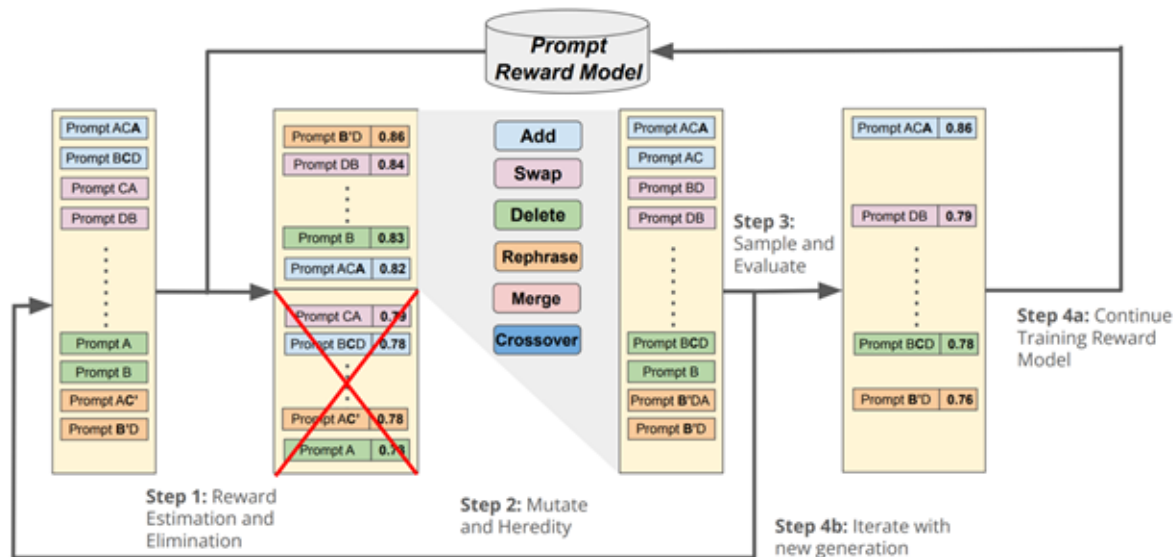


SPRIG

- **Motivation**
 - Prior prompt optimization mostly targets task-specific prompts
 - System prompts are more general, but less studied as an optimization target
- **Key Idea**
 - **SPRIG optimizes general-purpose system prompts using an edit-based genetic algorithm**
 - It constructs prompts from **reusable components** and evaluates them across many tasks
- **Takeaway**
 - A system prompt can be optimized as a general instruction space

SPRIG

- Generate candidate system prompts through edits, mutation, and crossover.
- Select prompts that improve performance across diverse tasks.
- Optimized system prompts can complement task-specific prompt optimization



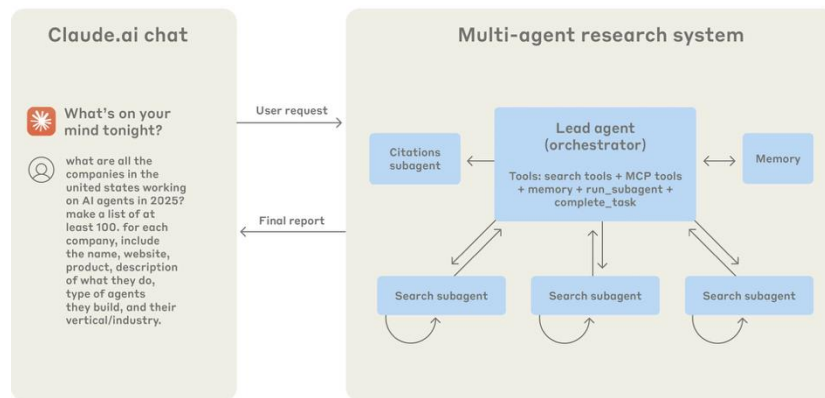
5. Prompts for Multi Agent System (MAS)

Anthropic's Multi-Agent Research System

● Key Idea

- **Orchestrator-subagent** architecture: a Lead Agent decomposes the query and dispatches **parallel subagents**
- Each subagent has its own context, tools, and trajectory; results are condensed via shared memory and synthesized by the lead

High-level Architecture of Advanced Research



Principles learned for prompting Multi-Agent

1. Think like the agent

Mentally simulate agents step-by-step to spot failure modes.

2. Teach the orchestrator to delegate

Subagents need objective, output format, tools, and clear boundaries.

3. Scale effort to query complexity

Embed rules: 1 agent for simple, 10+ for complex queries.

4. Tool design and selection

Clear, distinct tool descriptions; match tool to user intent.

5. Let agents improve themselves

Claude can diagnose failures and rewrite its own prompts.

6. Start wide, then narrow down

Explore the landscape with broad queries before drilling in.

7. Guide the thinking process

Extended thinking acts as a controllable planning scratchpad.

8. Parallel tool calling

3-5 subagents x 3+ tools each cuts research time by up to 90%.

Multi-Agent: Results & Trade-offs

● Results

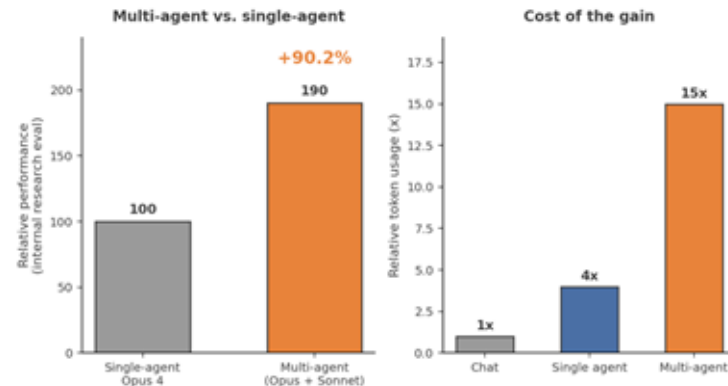
- Multi-agent (Opus 4 + Sonnet 4) outperforms single-agent Opus 4 by **+90.2%** on internal research eval
- Parallelization cuts research time by up to 90% for complex queries

● Cost of the gain

- **~15x more tokens** than a chat; single agents use ~4x
- Best fit: open-ended, breadth-first tasks where value justifies token cost

● Takeaway

- Multi-agent buys breadth and parallelism, at the price of token cost and coordination complexity



Tips! Best practices for great prompts

Prompt Structure

1. 1-2 sentences to establish role and high level task description
2. Dynamic / retrieved content
3. Detailed task instructions
4. Examples/ n-shot (optional)
5. Repeat critical instructions (particularly useful for very long prompts)

```
USER
You are an AI travel agent tasked with creating a personalized travel itinerary
based on the user's input. Your goal is to create an engaging, well-structured, and
realistic itinerary that matches the user's preferences and the specified location
and duration of the trip.

You will be provided with the following information:

<location>
{{LOCATION # }}
</location>

<num_days>
{{NUM_DAYS # }}
</num_days>

<user_preferences>
{{USER_PREFERENCES # }}
</user_preferences>

When creating the itinerary, follow these guidelines:
1. Research the location and its popular attractions, considering the user's
preferences.
2. Plan activities for each day, ensuring a good balance between sightseeing,
relaxation, and local experiences.
3. Include recommendations for meals, considering local cuisine and the user's
dietary preferences if mentioned.
4. Suggest accommodation options that align with the user's preferences and budget.
5. Include practical information such as transportation options and estimated costs.
6. Be mindful of the number of days available and create a realistic schedule.

Present your itinerary in the following format:
1. Start with a brief introduction to the destination.
2. Provide a day-by-day breakdown of activities, meals, and accommodations.
3. End with additional tips or recommendations for the trip.

Here's an example of how your output should be structured:

<example>
# EXAMPLE REMOVED FOR SCREENSHOT
</example>

Now, based on the provided location, number of days, and user preferences, create a
personalized travel itinerary. Be creative and thorough in your recommendations, and
ensure that the itinerary reflects the user's interests and the unique aspects of
the destination. Present your itinerary within <itinerary> tags.
```

Welcome to MLV!!



MLV Lab



MLV TV